

On a Grover-based quantum signature scheme and a teleportation-based design

Guoliang Xu ^{1*}, Songyao Xue ¹, Xiangfu Zou ^{2,*} and Yumei Zhang ³

¹ School of Software, Luoyang Normal University, Luoyang, 471934, China; xu1guo2liang@foxmail.com

² School of Mathematics and Computational Science, Wuyi University, Jiangmen, 529020, China

³ Conservatory of Music, Luoyang Normal University, Luoyang, 471934, China

* Correspondence: xu1guo2liang@foxmail.com, xf.zou@hotmail.com

Abstract

Quantum computation, with tools including Grover's algorithm, quantum walks, and quantum teleportation, plays an important role in quantum signature designs. Although such designs offer signature functionality, they sometimes come at the cost of security. This paper first reviews a Grover-based scheme and shows how man-in-the-middle attacks enable complete key recovery, Alice's disavowal, and Bob's forgery. We then propose an arbitrated quantum signature scheme based on quantum teleportation and a strengthened quantum one-time pad. The scheme is designed for quantum messages with known classical descriptions, not for arbitrary unknown quantum states. The security of the proposed scheme is analyzed through a formal adversarial model for unforgeability and non-repudiation, and its resilience against replay, intercept-and-resend, man-in-the-middle, entanglement, and collective or coherent attacks is examined. Numerical simulations and an asymptotic resource analysis further illustrate the practicality of the scheme.

Keywords: Quantum computation; Grover algorithm; Quantum signature; Security analysis

1. Introduction

In the field of quantum computation and communication, Grover's algorithm, quantum teleportation and quantum walk have inspired a variety of cryptographic constructions far beyond their original application scenarios [1–7]. Leveraging quantum superposition, phase manipulation and iterative evolution, these algorithm-driven schemes feature concise and flexible architecture, and offer security guarantees based on quantum information-theoretic principles. Among these quantum primitives, Grover's algorithm, as a fundamental building block in quantum computation and complexity theory, is particularly intriguing and conceptually appealing, offering a novel perspective for the design and construction of quantum signature schemes.

Among quantum signature paradigms, arbitrated quantum signature (AQS) schemes introduce a trusted or semi-trusted arbitrator to assist verification and dispute resolution [8–10]. Since the early scheme of Zeng and Keitel [8], a range of AQS constructions have been proposed using entanglement, Bell states, or quantum one-time pad (QOTP) encryption [9, 11–13]. A number of schemes built from seemingly sound quantum primitives were later shown to admit forgery, repudiation, or impersonation attacks [14–16]. These results suggest that the security of quantum signature schemes depends not only on the local soundness of the primitives employed, but also on the way those primitives are combined at the protocol level.

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Entropy* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

In 2015, Yoon et al. [4] proposed a quantum signature scheme that adopts Grover-type operations for signature generation and verification. Later in 2023, Yoon et al. [6] further extended this line of research by presenting a quantum asymmetric key cryptographic scheme based on Grover iteration. Their work migrated Grover-related operations from quantum signature scenarios to the domain of quantum asymmetric encryption, further enriching the application boundaries of Grover primitives in quantum cryptography. From the algorithmic perspective, such designs are conceptually appealing since they embed standard quantum search routines directly into cryptographic workflows. Nevertheless, combining reversible quantum transformations with publicly accessible verification parameters leads to an inherent structural conflict: information that is harmless in standard quantum search scenarios may become a critical security vulnerability in signature and encryption systems.

In this paper, we study quantum signature security from that structural perspective. Our main contributions are as follows:

- (1)

We analyze the Grover-based quantum signature scheme of Yoon *et al.*[4] and demonstrate that it is vulnerable to man-in-the-middle attacks, which enable complete key recovery, Alice’s disavowal, and Bob’s existential forgery.
- (2)

We propose an arbitrated quantum signature scheme based on quantum teleportation and a strengthened QOTP [16]. The scheme is designed for quantum messages with known classical descriptions, not for arbitrary unknown quantum states. A formal security analysis under a well-defined adversarial model shows that the scheme satisfies unforgeability and non-repudiation, and numerical simulations illustrate its practical viability.

The remainder of this paper is organized as follows. Section 2 introduces the necessary preliminaries. Section 3 reviews and analyzes the Grover-based quantum signature scheme of Yoon *et al.*[4]. Section 4 proposes and analyzes an arbitrated quantum signature scheme that combines quantum teleportation and strengthened QOTP [16]. Section 5 provides a resource overhead analysis of the proposed scheme and YKLY’2015, and simulations of the attacks on the proposed scheme. Finally, Section 6 concludes the paper. The complete simulation codes are provided in the Appendix.

2. Preliminaries

This section introduces the notation, quantum primitives, and background used throughout the paper.

To begin with, for the reader’s convenience, the principal symbols used throughout the paper are summarized in Table 1.

Table 1. Summary of key notation.

General		Primitives		Protocol	
$\mathbb{C}$	Complex numbers	$\delta_x, \delta_y, \delta_z$	Pauli operators	$K_{AT}(16)$	Key (Alice-Trent)
i	Imaginary unit	F	State fidelity	$K_{BT}(24)$	Key (Bob-Trent)
$\cong$	Equal up to phase	$ p\rangle$	Message state	$K'_{AT}(4)$ etc.	Key segments
α	Physical error rate	$E_K p\rangle$	Strengthened QOTP	X_1, X_2, X_3	Bell outcomes
λ	Security parameter	$E_k(\rho)$	Standard QOTP	$ p_1\rangle, p_2\rangle, p_3\rangle$	Message copies
K^i	i -th bit of K	$U(X)$	Pauli operation	$ p'_2\rangle, p'_3\rangle, X'_1$	Forged values
$ X\rangle$	State for n -bit X	$ \varphi^\pm\rangle, \psi^\pm\rangle$	Bell states	p_{dep}	Depolarizing noise
$t_{\text{comm}}, t_{\text{op}}$	Latency, local time	$ \varphi^+\rangle_{AB}, \varphi^+\rangle_{BT}$	Shared Bell pairs		

Then, Grover’s algorithm provides a quadratic speedup for unstructured search by iteratively amplifying the amplitude of marked states [5]. In Grover-based quantum signature schemes [4], the mechanism of unitary operations is represented by $U_S = I - 2|K\rangle\langle K|$, $U_V = 2|M\rangle\langle M| - I$, and their combined action $U_V U_S|M\rangle = a|K\rangle$. Here, applying

the unitary operators U_S and U_V , this mechanism can be used to identify the desired data $|K\rangle$ (for example $|K\rangle = |00\rangle$) from database $|M\rangle$ (for example, $|M\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$). The Grover-based quantum signature scheme [4] was proposed by utilizing the unitary operations U_V and U_S .

The teleportation procedure used in our protocol proceeds as follows. Alice combines the message state $|p\rangle = \alpha|0\rangle + \beta|1\rangle$ ($\alpha, \beta \in \mathbb{C}, |\alpha|^2 + |\beta|^2 = 1$) with the Bell state $|\varphi^+\rangle_{AB}$ to obtain a three-particle entangled state

$$|p\rangle_A \otimes |\varphi^+\rangle_{AB} = \frac{1}{2} [|\varphi^+\rangle_{AA}(\alpha|0\rangle + \beta|1\rangle)_B + |\varphi^-\rangle_{AA}(\alpha|0\rangle - \beta|1\rangle)_B + |\psi^+\rangle_{AA}(\alpha|1\rangle + \beta|0\rangle)_B + |\psi^-\rangle_{AA}(\alpha|1\rangle - \beta|0\rangle)_B]. \quad (1)$$

Defining $U(X)$ as

$$U(X) = \begin{cases} I & = |0\rangle\langle 0| + |1\rangle\langle 1|, X = 00, \\ \delta_x & = |1\rangle\langle 0| + |0\rangle\langle 1|, X = 01, \\ \delta_z & = |0\rangle\langle 0| - |1\rangle\langle 1|, X = 10, \\ \delta_z\delta_x & = |0\rangle\langle 1| - |1\rangle\langle 0|, X = 11. \end{cases} \quad (2)$$

the above state $|p\rangle_A \otimes |\varphi^+\rangle_{AB}$ can be compactly written as

$$|p\rangle_A \otimes |\varphi^+\rangle_{AB} = \frac{1}{2} \sum_{X=00}^{11} [I \otimes U(X)] |\varphi^+\rangle_{AA} \otimes U(X) |p\rangle_B, \quad (3)$$

where each $[I \otimes U(X)] |\varphi^+\rangle$ is one of the four Bell states. Alice then performs a Bell measurement on her two particles. The state collapses to

$$[I \otimes U(X)] |\varphi^+\rangle_{AA} \otimes U(X) |p\rangle_B, \quad (4)$$

with X uniformly distributed over $\{00, 01, 10, 11\}$. Alice obtains X from the Bell measurement outcome, while Bob's particle collapses to $U(X)|p\rangle$. We refer to this process as Alice teleporting $U(X)|p\rangle$ to Bob.

The strengthened quantum one-time pad (QOTP) proposed by Kim *et al.* [16] addresses forgeability issues present in standard QOTP-based arbitrated quantum signature schemes [14,15]. Using a 4-bit key K , a single-qubit message $|p\rangle = \alpha|0\rangle + \beta|1\rangle$ is encrypted as

$$|q\rangle = E_K |p\rangle = \delta_x^{K^1} \delta_z^{K^2} T \delta_x^{K^3} \delta_z^{K^4} |p\rangle, \quad (5)$$

where $T = \frac{i}{\sqrt{3}}(\delta_x - \delta_y + \delta_z)$. The standard QOTP achieves information-theoretic security when keys are uniformly random and never reused [17,18]. Our protocol inherits this property by generating fresh keys for each execution.

3. Analysis of a Grover-based quantum signature scheme (YKLY'2015)

In this section, we analyze the quantum signature scheme based on Grover's search algorithm proposed by Yoon, Kang, Lim, and Yang (YKLY'2015) [4]. We show that the scheme suffers from fundamental structural vulnerabilities that lead to complete key recovery and universal forgery.

3.1. Review of a Grover-based quantum signature scheme (YKLY'2015)

In the interest of readability, we briefly review YKLY'2015 [4] in this section. The scheme consists of two communication processes: between Alice and TC, and between TC and Bob. Meanwhile, both of the two processes consist of three phases: preparation, signature and verification. YKLY'2015 [4] can be described as follows.

3.1.1. Process between Alice and TC

In the Preparation phase, there are the following two steps.

(AT1) Alice announces the classical message m that she wants to send to Bob on the public board. Then, Alice and TC share the secret key K_{AT} by a QKD protocol, such as the QKD protocols [19–21], which provides unconditional security [22,23].

(AT2) Alice creates the signing unitary operator $U_{S_{AT}} = I - 2|K_{AT}\rangle\langle K_{AT}|$ where $|K_{AT}\rangle$ is a two-qubit state generated from the classical two-bit key K_{AT} . For example, $|K_{AT}\rangle = |ij\rangle$ if $K_{AT} = ij$ for all $i, j \in \{0, 1\}$.

In the Signature phase, Alice makes her signature in Step (AT3) and sends it to TC by Steps (AT4)–(AT7).

(AT3) Using the classical message m , Alice generates an original message qubit $|M\rangle$ where $|M\rangle = |-\rangle \otimes |-\rangle$ if $m = 00$, $|M\rangle = |-\rangle \otimes |+\rangle$ if $m = 01$, $|M\rangle = |+\rangle \otimes |-\rangle$ if $m = 10$, and $|M\rangle = |+\rangle \otimes |+\rangle$ if $m = 11$. Then, she obtains the signature $U_{S_{AT}}|M\rangle = (I - 2|K_{AT}\rangle\langle K_{AT}|)(|M\rangle)$ using $|M\rangle$ and $U_{S_{AT}}$.

(AT4) Alice includes decoy qubits of the EPR pair, $|\psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$, in random locations among the message qubits $U_{S_{AT}}|M\rangle$. Alice sends TC the first sequence of particles of the message qubits, and tells TC the locations of half the decoy qubits.

(AT5) TC performs measurement on the decoy qubits using Z or X basis, and tells Alice his measurement bases and measurement outcomes. Alice uses the same measuring bases to measure the corresponding qubits in the sequence and compares the measurement outcomes with TC's. If the error rate is smaller than the channel average error rate, Alice and TC conclude that there are no eavesdroppers in the channel and continue the protocol.

(AT6) Alice performs random unitary operations chosen from $\{I, X, Y, Z\}$ on the other half of the decoy qubits. She sends the other sequence of the particles to TC, and informs TC the positions of decoy qubits and the types of unitary operations she has chosen.

(AT7) TC confirms the security of the communication channel based on the received information.

In the Verification phase, there is only one step.

(AT8) TC first makes the verifying unitary operator $U_V = 2|M\rangle\langle M| - I$ based on the known message m . Then, TC implements U_V on the received message to recover the shared key K_{AT} as $U_V U_{S_{AT}}|M\rangle = \pm|K_{AT}\rangle$. Finally, TC compares the result of $U_V U_{S_{AT}}|M\rangle$ with the shared key. If they are identical, TC confirms that Alice is a legitimate signer.

3.1.2. Process between TC and Bob

Confirming that the received message was legitimately signed by Alice, TC sends the message, his signature, to Bob by repeating the same processes above. By this process, Bob can ensure that the message received from Alice, in fact, the channel, is signed by her.

In the Preparation phase, there are the following two steps.

(TB1) TC and Bob share the secret key K_{TB} by a QKD protocol, such as the QKD protocols [19–21], which can provide unconditional security [22,23].

(TB2) TC makes the signing unitary operator $U_{S_{TB}} = I - 2|K_{TB}\rangle\langle K_{TB}|$ where $|K_{TB}\rangle$ is a two-bit state generated from K_{TB} as with K_{AT} .

In the Signature phase, TC makes his signature in Step (TB3) and sends it to Bob by Steps (TB4)–(TB7).

(TB3) TC signs the message $|M\rangle$ using $U_{S_{TB}}$ as $U_{S_{TB}}|M\rangle = (I - 2|K_{TB}\rangle\langle K_{TB}|)(|M\rangle)$.

(TB4) TC includes decoy qubits of the EPR pair, $|\psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$, in random locations among the message qubits $U_{S_{TB}}|M\rangle$. TC sends Bob the first sequence of particles of the message qubits, and informs Bob of the locations of half the decoy qubits.

(TB5) Bob performs measurement on the decoy qubits using Z or X basis, and tells TC his measurement bases and measurement outcomes. TC uses the same measuring bases to

measure the corresponding decoy qubits in the sequence and compares the measurement outcomes with those of Bob. If the error rate is smaller than the channel average error rate, TC and Bob conclude that there are no eavesdroppers in the channel, and continue the protocol.

(TB6) TC performs unitary operations chosen from $\{I, X, Y, Z\}$ on the other half of the decoy qubits. TC sends the other sequence of particles to Bob, and informs him the positions of decoy qubits and the types of unitary operations he has chosen.

(TB7) Bob confirms the security of communication channel based on the information. In the Verification phase, there is only one step.

(TB8) Bob makes the verifying unitary operator $U_V = 2|M\rangle\langle M| - I$ based on the known message m . And then, Bob performs U_V on the received message to recover the shared key K_{TB} as $U_V U_{S_{TB}}|M\rangle = \pm|K_{TB}\rangle$. Bob compares the result of $U_V U_{S_{TB}}|M\rangle$ with the shared key. If they are identical, Bob confirms that TC is a legitimate signer.

3.2. Man-in-the-middle (MITM) attacks

In YKLY'2015 [4], TC plays a crucial role including the arbitrator. When a dispute arises between Alice and Bob, only TC has the right to dispute resolution. In this section, we first give a MITM attack on YKLY'2015 [4]. Then, we show how Alice uses the proposed MITM attack to disavow signature and Bob uses the proposed MITM attack to forge signature.

3.2.1. Eve's MITM attack

The MITM attack on YKLY'2015 [4] consists of three processes: between Alice and Eve, between Eve and TC, and between TC and Bob. In this attack, Eve completely intercepts and replaces the quantum channel between Alice and TC, impersonating TC to Alice and Alice to TC. The attack is illustrated in Figure 1 and we present it in detail as follows.

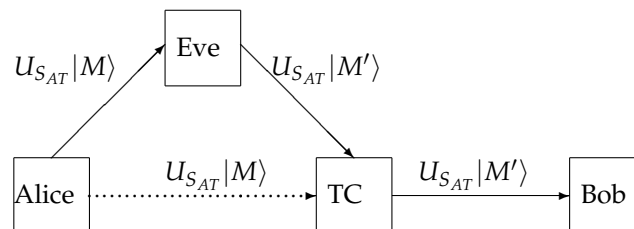


Figure 1. MITM attack. Solid lines correspond to the process of the attack while the dotted line denotes the corresponding process of YKLY'2015 [4].

In the Process between Alice and Eve, the attacker Eve first intercepts the message qubits $U_{S_{AT}}|M\rangle$ to obtain the shared key K_{AT} by pretending to be TC and performing the following four steps.

(AE1) In Step (AT4), Eve intercepts the first sequence of particles of the message qubits $U_{S_{AT}}|M\rangle$ and the message that Alice tells TC the locations of half the decoy qubits.

(AE2) In Step (AT5), Eve performs measurement on the decoy qubits using Z or X basis, and impersonates TC to tell Alice her measurement bases and measurement outcomes.

(AE3) In Step (AT6), Eve intercepts the other sequence of the particles that Alice sends to TC and the message that Alice tells TC the positions of decoy qubits and the types of unitary operations.

(AE4) In Step (AT8), Eve makes the unitary operator $U_V = 2|M\rangle\langle M| - I$ based on the known message. Then, she implements U_V on the received message to recover the shared key as $U_V U_{S_{AT}}|M\rangle = \pm|K_{AT}\rangle$. Thereby, Eve can obtain the shared key K_{AT} successfully.

In the Process between Eve and TC, Eve performs the following two steps to pass TC's verify by posing as Alice.

(ET1) Eve pretends to be Alice to announce forged message m' and makes its signature $U_{S_{AT}}|M'\rangle$ with K_{AT} .

(ET2) Eve impersonates Alice to communicate with TC by performing Steps (AT4)-(AT8).

Finally, there is nothing Eve needs to do in the process between TC and Bob.

By performing the MITM attack above, $U_{S_{AT}}|M'\rangle$ as the signature of the forged message m' can pass TC's verifying. Therefore, Eve can forge Alice's signature for any message successfully.

Remark 1. Similarly, to forge Alice's signature in YKLY'2015 [4], Eve can perform an MITM attack between TC and Bob: Eve intercepts the message qubits $U_{S_{TB}}|M\rangle$ to obtain the shared key K_{TB} , pretends to be Alice to announce forged message m' , makes the forged signature $U_{S_{TB}}|M'\rangle$ with K_{TB} , and sends $U_{S_{TB}}|M'\rangle$ to Bob as TC in YKLY'2015.

3.2.2. Alice's disavowal attack

Alice can achieve her disavowal in YKLY'2015 [4] by performing an MITM attack between TC and Bob which consists of two processes: between TC and Alice, and between Alice and Bob. The MITM attack is described in Figure 2. Its details will be shown in the

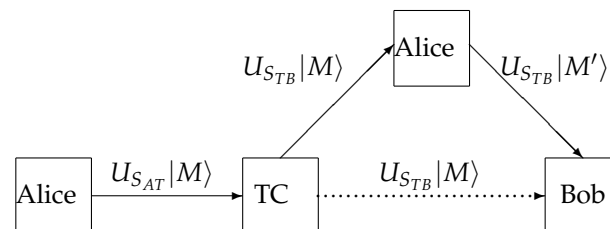


Figure 2. Alice's disavowal attack. Solid lines correspond to the process of the attack while the dotted line denotes the corresponding process of YKLY'2015 [4].

following.

In the Process between TC and Alice, Alice first pretends to be Bob and performs the following four steps.

(TA1) In Step (TB4), Alice intercepts the first sequence of particles of the message qubits $U_{S_{TB}}|M\rangle$ and the message that TC tells Bob the locations of half the decoy qubits.

(TA2) In Step (TB5), Alice performs measurement on the decoy qubits using Z or X basis, and impersonates Bob to tell TC her measurement bases and measurement outcomes.

(TA3) In Step (TB6), Alice intercepts the other sequence of the particles that TC sends to Bob and the message that TC tells Bob the positions of decoy qubits and the types of unitary operations.

(TA4) In Step (TB8), Alice makes a unitary operator $U_V = 2|M\rangle\langle M| - I$ based on the known message. She implements the operator U_V on the received message to recover the shared key as $U_V U_{S_{TB}}|M\rangle = \pm|K_{TB}\rangle$. So far, Alice has got the shared key K_{TB} .

In the Process between Alice and Bob, Alice poses as TC to perform the following two steps to let Bob accept her forged signature.

(AB1) Alice announces the forged message m' and makes the forged TC's signature $U_{S_{TB}}|M'\rangle$ with K_{TB} .

(AB2) Alice communicates with Bob to perform Steps (TB4)-(TB8) by posing as TC.

By performing operations above, Alice can forge a message and its signature which can pass Bob's verifying. Note that the sources of the messages received by Bob cannot be distinguished. In fact, the message which TC has verified and the message which Bob has received are not the same one in Alice's attack. As a result, Alice can announce that the m is not the initial message to achieve her benefit.

3.2.3. Bob's forgery attack

Bob's forgery attack follows the same pattern as Eve's MITM attack, applied to the TC-Bob phase. As the legitimate receiver, Bob already possesses the correct position to intercept the message qubits $U_{S_{TB}}|M\rangle$ sent from TC to Bob in Steps (TB4)-(TB7). Bob extracts the shared key K_{TB} using the publicly known verifying operator $U_V = 2|M\rangle\langle M| - I$, which he can construct from the message m announced on the public board. With K_{TB} in hand, Bob can sign any forged message m' by preparing $U_{S_{TB}}|M'\rangle$ and present it as evidence that the message originated from Alice and was verified by TC. Since TC's verification in the TC-Bob phase only confirms that a message was signed with K_{TB} , Bob's forged signature passes verification. Consequently, Bob can successfully forge Alice's signature for any message of his choice.

3.3. The main reasons why YKLY'2015 is insecure

In a signature scheme, TC should be able to judge the validity of the signature and arbitrate the dispute between Alice and Bob. However, by the attacks in Section 3.2, it is not the case in YKLY'2015 [4]. In this section, we discuss the reasons why YKLY'2015 [4] is vulnerable to MITM attacks.

First, according to YKLY'2015 [4], we can know that the process between Alice and TC is used to confirm that the received message m was legitimately signed by Alice or not. That is, in Step (AT1), TC cannot confirm that the message in public board (such as the initial message m and the forged message m') really comes from Alice or not. Otherwise, the subsequent steps in the scheme [4] are unnecessary because the message in public board has been authenticated by TC. In other words, the public board cannot provide authentication so that Eve can announce some messages in public board.

Meanwhile, the required information, including data such as the sender's quantum signatures and the positions of decoy states, is transmitted over a public channel without any prior encryption. While the protocol specifies that the sender delivers messages to the trusted center (TC), it lacks supplementary safeguards to ensure exclusive reception by the TC. Neither does the protocol guarantee that unauthorized third parties that intercept the data cannot extract meaningful valuable information. It is critical to note that although quantum key distribution (QKD) provides information-theoretic unconditional security, this security guarantee only applies to the key generation process, not to the encryption of quantum signature data during transmission.

Thereby, in order to judge which one of m and m' in public board is Alice's message, TC has no alternative but to recover the secure key from $U_{S_{AT}}|M\rangle$ or $U_{S_{AT}}|M'\rangle$. For the same reason, Alice isn't able to ensure the following two cases: (1) The receiver of the message that Alice tells to TC is indeed TC; (2) The message received by Alice really comes from TC. This explains why Eve can intercept the signature $U_{S_{AT}}|M\rangle$ during transmission without being detected, as the protocol provides no authentication of the communicating parties.

A critical vulnerability lies in the fact that the verifying unitary operator $U_V = 2|M\rangle\langle M| - I$ depends entirely on the message m , which is publicly announced on the board. Since m is known to all parties, anyone, including an eavesdropper, can construct U_V independently. This means that the verification procedure provides no authentication: the ability to verify a signature is not restricted to the intended recipient but is available to any adversary who reads the public board. Combined with the unprotected transmission of the signature $U_{S_{AT}}|M\rangle$, this allows Eve to recover the secret key K_{AT} by simply applying U_V to the intercepted signature.

TC is only able to confirm that the received message $U_{S_{AT}}|M'\rangle$ was signed using the shared key. So, Eve can prove to TC that he is Alice. Since TC can only verify that a message

was signed with K_{AT} , and Eve has recovered K_{AT} , Eve can successfully impersonate Alice to TC. This same vulnerability also exists in the TC-Bob phase. Similarly, the phase between TC and Bob also faces the same problems. As a result, TC's confirmation doesn't provide message authentication and non-repudiation. In addition, message integrity cannot be guaranteed by the signature key because Eve can obtain the secret key to sign any message. Therefore, none of the three basic conditions is satisfied in YKLY'2015 [4].

4. An AQS scheme on Teleportation-QOTP

In this section, we present an arbitrated quantum signature (AQS) scheme designed to avoid the weaknesses identified in the Grover-based design. We first define the system model and assumptions, then describe the protocol, and finally provide a formal security analysis.

4.1. System model and assumptions

The proposed scheme is designed for an arbitrated setting where two mutually distrusting parties require a trusted arbitrator to resolve disputes. The following model formalizes the participants, their trust relationships, the communication channels, the adversary capabilities, and the security properties that the protocol must satisfy.

4.1.1. Participants and main quantum primitives

The protocol involves three parties. Alice is the signer, who holds a quantum message $|p\rangle$ whose classical description is known to her, and wishes to generate a valid signature. Bob is the receiver and verifier, who authenticates the signed message with the assistance of the arbitrator. Trent is the arbitrator, who is trusted to facilitate signature generation, verification, and dispute resolution.

The quantum message to be signed, denoted by $|p\rangle$, is assumed to be a state whose classical description is known to Alice. Consequently, Alice can prepare multiple identical copies of $|p\rangle$ without violating the no-cloning theorem. This assumption is adopted throughout the protocol and distinguishes our scheme from signature protocols designed for completely unknown quantum states.

Our protocol employs three main quantum primitives: the strengthened quantum one-time pad for encrypting quantum states (see Section 2 for details), quantum teleportation for transferring quantum states (see Section 2 for details), and the swap test for comparing quantum states during the verification phase, which we now describe in more detail.

The swap test is a well-established probabilistic procedure for comparing unknown quantum states that has been widely adopted in quantum information processing. It takes two states $|\psi\rangle$ and $|\phi\rangle$ as input, and applies a controlled-SWAP gate between them, controlled on an auxiliary qubit initialized in $|+\rangle$, followed by a Hadamard gate on the auxiliary qubit and a measurement in the computational basis. When the two states are identical, the test always outputs "equal". When they differ, the test outputs "equal" with probability $(1 + F)/2$ and "not equal" with probability $(1 - F)/2$, where $F = |\langle\psi|\phi\rangle|^2$ denotes the fidelity between the two states. For orthogonal states ($F = 0$), this reduces to $1/2$ each. By repeating the test m times on independent copies, the probability that all m tests erroneously output "equal" for two different states is $[(1 + F)/2]^m$. For orthogonal states ($F = 0$), this reduces to $(1/2)^m = 2^{-m}$.

In a practical protocol, the participants fix a fidelity threshold $F_{\max}$, which captures the maximum fidelity that a modification may achieve without producing a semantically meaningful alteration of the message. Any meaningful forgery must reduce the fidelity to at most $F_{\max}$, i.e., $F \leq F_{\max}$. The false-acceptance probability after λ repetitions is then bounded by $[(1 + F_{\max})/2]^\lambda$, and the detection probability is at least $1 - [(1 + F_{\max})/2]^\lambda$.

The threshold $F_{\max}$ is chosen according to the application scenario, in the same way that the security parameter λ is chosen according to the required security level. For instance, in a contract-signing setting, a modification that changes only a negligible portion of the contract does not constitute a meaningful forgery, just as in classical contract law a trivial typographical change does not invalidate the substance of the agreement. If the application demands the strictest verification, one may set $F_{\max} = 0$, in which case the bound reduces to $2^{-\lambda}$.

The resource cost of a single swap test is modest: it requires one auxiliary qubit, one controlled-SWAP gate per pair of compared qubits, and one Hadamard gate, plus a single-qubit measurement. For single-qubit states, these operations are constant. For n -qubit registers, the number of controlled-SWAP gates scales as $O(n)$, while the auxiliary qubit and measurement overhead remain constant. When the test is repeated $m = \lambda$ times, the total resource cost scales linearly with λ . In addition, each repetition consumes the two input copies, so λ repetitions require λ independent physical copies of each state being compared. In the proposed protocol, every occurrence of a message copy is therefore understood as a message group consisting of the required number of physical copies. Specifically, the logical copy $|p_1\rangle$ is realized as $2\lambda + 1$ physical copies, while the logical copies $|p_2\rangle$ and $|p_3\rangle$ are each realized as λ physical copies. All $2\lambda + 1$ physical copies of $|p_1\rangle$ are transmitted to Trent through the two-stage teleportation chain via Bob. Among them, λ copies are consumed by Trent in the λ swap tests during the intermediate verification, and the remaining $\lambda + 1$ copies are teleported back to Bob for the final verification. The λ copies of $|p_3\rangle$ serve as the arbitrator's reference copies, and the λ copies of $|p_2\rangle$ serve as the receiver's reference copies. All these copies are prepared by the signer, which is possible because the signer knows the classical description of the message. In total, Alice prepares $4\lambda + 1$ physical copies of the message. The λ copies of $|p_2\rangle$ are transmitted in plaintext to Bob as reference copies, whereas the λ copies of $|p_3\rangle$ are encrypted with the strengthened QOTP and transmitted to Trent. The remaining $2\lambda + 1$ physical copies of $|p_1\rangle$ are transmitted through the teleportation chain from Alice to Bob and then to Trent. The first teleportation hop consumes $2\lambda + 1$ Bell pairs, the second hop consumes $2\lambda + 1$ Bell pairs, and the final hop from Trent back to Bob consumes $\lambda + 1$ Bell pairs, giving $5\lambda + 3$ Bell pairs in total. Accordingly, the QOTP encryption cost and the direct communication cost each scale as $O(\lambda n)$, the teleportation cost requires $(5\lambda + 3)n$ Bell pairs and therefore scales as $O(\lambda n)$, and the swap-test circuit cost scales as $O(\lambda n)$ for an n -qubit message. Since λ is fixed once the security level and the fidelity threshold are chosen, all these costs are constant multiples of the logical message size, and the asymptotic complexity remains $O(n)$. In the protocol description, the swap test between two logical copies is understood as λ independent swap tests between the corresponding physical copies in their message groups. For brevity, the protocol description omits these physical copy details and refers to each logical message copy directly.

In cryptographic protocols, it is standard practice to anchor all security-critical parameters to a single security parameter, typically denoted by λ , which quantifies the overall security strength of the system. The defining property is that any adversary should succeed with probability at most $2^{-\lambda}$, which is a negligible function in λ . Following this principle, we set the repetition count of the swap test to $m = \lambda$. With the fidelity threshold $F_{\max}$, the corresponding false-acceptance bound is $[(1 + F_{\max})/2]^\lambda$, which reduces to $2^{-\lambda}$ when $F_{\max} = 0$. The same parameter λ governs the length of pre-shared authentication keys and the number of bits consumed by the quantum one-time pad, so that every component of the protocol operates at a consistent security level. For practical deployment, the value of λ can be chosen according to the required security level.

4.1.2. Trust model

Trent is trusted. The arbitrator follows the protocol honestly and does not collude with either Alice or Bob. Trent is responsible for key distribution, signature verification, and dispute resolution. This assumption reflects the role of trusted institutions such as notaries, certification authorities, and judicial bodies in conventional legal and commercial frameworks. In such settings, the trusted third party is assumed to be impartial and bound by procedural rules, which makes the assumption both natural and well-motivated. We further assume that Trent does not collude with either party individually. If Trent were to collude with Alice, he could help her repudiate a valid signature. If Trent were to collude with Bob, he could help him forge a signature. Both scenarios would render the arbitrated model meaningless and are therefore excluded from our security analysis.

Alice and Bob are mutually distrusting. Each party may attempt to cheat the other. Alice may try to repudiate a signature she has generated, while Bob may attempt to forge a signature or alter a signed message. This assumption models realistic adversarial behavior in contractual relationships, where neither party has reason to trust the other's honesty and each acts in their own interest.

No collusion between Alice and Bob. We assume that Alice and Bob do not collude jointly to defraud the arbitrator or to compromise the protocol. This is a standard and well-motivated assumption in arbitrated signature schemes. If Alice and Bob were willing to collude, they would be acting in concert rather than in conflict, and the protocol would no longer be needed. The purpose of an arbitrated signature is to protect each party against a potentially dishonest counterparty. When both parties choose to cooperate, there is simply no dispute for the arbitrator to resolve, and the scenario falls outside the intended scope of the model.

The following example illustrates a realistic setting in which all three assumptions of our trust model naturally hold. Consider two multinational corporations, Alpha Corporation and Beta Limited, that have no prior business relationship and wish to sign a high-value procurement contract remotely. Neither party fully trusts the other: Alpha Corporation worries that Beta Limited may later deny having signed the contract or alter its terms, while Beta Limited worries that Alpha Corporation may forge its signature on a different document. To resolve this mutual distrust, both parties agree to use an internationally recognized third-party platform, such as a global bank or a cross-border digital notary service, to arbitrate the signing process. In our scheme, this platform plays the role of Trent. The signer executes the proposed protocol, and the platform verifies that the quantum signature is consistent with the signer's identity and the contract content before forwarding the verified message to the receiver. If a dispute arises later, the platform serves as the authoritative witness because it has already verified the authenticity and integrity of the signed contract. This example satisfies all three assumptions of our trust model: (i) the platform is an established institution with legal obligations to remain impartial, so Trent is trusted and does not collude with either party; (ii) Alpha Corporation and Beta Limited are mutually distrusting, as they have no established relationship and each has incentives to protect its own interests; and (iii) the two companies do not collude, because they are contractual counterparts with conflicting interests. This setting is realistic because existing cross-border commerce already relies on centralized intermediaries such as escrow services, correspondent banks, and notarial platforms to mediate transactions and provide certification. Such scenarios are widespread in practice, which demonstrates the broad applicability of the trusted-arbitrator model assumed in this work.

4.1.3. Channel assumptions

We assume that the classical channels between all parties are authenticated but not necessarily confidential. An adversary may observe the messages on these channels but cannot modify them without detection. The quantum channels are assumed to be fully insecure: an adversary may intercept, measure, or replace quantum states in transit. The security of our protocol does not rely on the confidentiality of the quantum channels. Except for the plaintext $|p_2\rangle$ reference copies transmitted from Alice to Bob, the protected quantum objects transmitted over insecure channels are encrypted using the strengthened QOTP. We also assume that quantum key distribution links are available between each pair of parties to establish the pre-shared keys K_{AT} and K_{BT} .

4.1.4. Adversary model

We consider two classes of adversaries. An external adversary is not a legitimate participant in the protocol and does not possess any pre-shared keys, but has full access to the classical and quantum channels and may eavesdrop on all classical messages, intercept quantum states in transit, inject forged quantum states or classical messages into the channels, capture a transmitted state and perform arbitrary quantum operations on it before forwarding the result to the intended recipient, and record and later retransmit previously observed messages or states. For the analysis of quantum-state confidentiality, the external adversary is assumed to be computationally unbounded but limited by the laws of quantum mechanics. For the analysis of unforgeability and non-repudiation, however, we restrict the adversary to be polynomial-time in the security parameter λ , because those properties rely on classical cryptographic primitives whose security is computational rather than information-theoretic. Internal adversaries consist of a dishonest Alice or a dishonest Bob, either of whom may deviate arbitrarily from the protocol specification. A dishonest Alice may attempt to repudiate a valid signature that she has genuinely generated by tampering with the quantum states she prepares, sending inconsistent classical messages, or exploiting the dispute resolution procedure. A dishonest Bob may attempt to forge a valid signature by manipulating the verification process, altering the quantum states he receives, or presenting modified evidence to Trent during dispute resolution. Both internal adversaries have full knowledge of the protocol specification and access to their own keys and quantum resources, but they do not possess the keys held by the other party or by Trent.

4.1.5. Security properties and formal security games

Under the above model, the proposed scheme aims to satisfy unforgeability and non-repudiation. We now define these properties through formal security games between a challenger and an adversary.

Unforgeability. Unforgeability requires that no adversary can generate a valid signature attributable to Alice without Alice's participation. We define the following game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$, where $\mathcal{A}$ may be an external adversary or a dishonest Bob. In the setup phase, $\mathcal{C}$ generates the pre-shared keys K_{AT} and K_{BT} and gives K_{BT} to $\mathcal{A}$. If $\mathcal{A}$ is an external adversary, it receives no keys. In the query phase, $\mathcal{A}$ may polynomially many times request Alice to sign messages of its choice. For each query with message $|p_i\rangle$, $\mathcal{C}$ runs the protocol on behalf of Alice and Trent and returns the resulting transcript to $\mathcal{A}$. In the forgery phase, $\mathcal{A}$ outputs a message $|p^*\rangle$ and a claimed signature σ^* , where $|p^*\rangle$ was never queried in the previous phase. In the verification phase, $\mathcal{C}$ runs the verification procedure on behalf of Trent. $\mathcal{A}$ wins if Trent outputs "valid" and attributes the signature to Alice. The scheme satisfies unforgeability if, for any polynomial-time adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins this game is a negligible function in λ , where λ is the

security parameter. The restriction to polynomial-time adversaries applies to unforgeability because the protocol's authentication components are computationally secure.

Non-repudiation. Non-repudiation requires that if Alice genuinely generates a valid signature, she cannot later deny having done so. We define the following game between a challenger $\mathcal{C}$ and a dishonest Alice $\mathcal{A}$. In the setup phase, $\mathcal{C}$ generates the pre-shared keys K_{AT} and K_{BT} and gives K_{AT} to $\mathcal{A}$, and also initializes Bob and Trent with their respective keys. In the signature generation phase, $\mathcal{A}$ follows the protocol honestly and generates a valid signature σ on a message $|p\rangle$ of her choice, with Bob and Trent both following the specification honestly. In the repudiation attempt phase, $\mathcal{A}$ outputs a claim that the signature σ is invalid, together with any evidence she wishes to present to the arbitrator. In the dispute resolution phase, $\mathcal{C}$ invokes Trent's dispute resolution procedure using the evidence provided by $\mathcal{A}$ and any records held by Bob and Trent. $\mathcal{A}$ wins if Trent concludes that the signature is invalid. The scheme satisfies non-repudiation if, for any polynomial-time dishonest Alice $\mathcal{A}$, the probability that $\mathcal{A}$ wins this game is a negligible function in λ , where λ is the security parameter. The restriction to polynomial-time adversaries applies to non-repudiation because the dispute-resolution records rely on computationally secure classical authentication.

4.2. The proposed scheme

To satisfy the security properties defined in Section 4.1, the proposed scheme combines entanglement distribution, quantum teleportation [24], and strengthened quantum one-time pad encryption [16]. The protocol consists of two phases, the initializing phase and the signing-verifying phase. For simplicity, the protocol is described for a single physical copy of the quantum message and a single Bell pair at each teleportation step. The description of the operations on one copy completely determines the operations on all other copies. In actual execution, the required numbers of message copies, Bell pairs, and classical key bits are supplied in sufficient quantities according to the consumption of each step, and these numbers are summarized after the protocol description.

4.2.1. Initializing phase

In this phase, the secret keys are shared and the Bell states are generated.

(I1) Alice and Bob share their secret keys $K_{AT} \in \{0, 1\}^{16}$, $K_{BT} \in \{0, 1\}^{24}$ with Trent by QKD protocols [19–21] respectively, which are proved to be unconditionally secure [25,26]. K_{AT} is divided into three segments: K'_{AT} (bits 1–4), K''_{AT} (bits 5–8), and K'''_{AT} (bits 9–16). Similarly, K_{BT} is divided into four segments: K'_{BT} (bits 1–4), K''_{BT} (bits 5–12), K'''_{BT} (bits 13–16), and $K^{(4)}_{BT}$ (bits 17–24). In practice, QKD can operate continuously in the background to maintain a pool of pre-shared keys. The keys required for each signature are then drawn from this pool, so that the key generation does not delay the signing process. The key-partition pattern specified above, namely the 16-bit K_{AT} and the 24-bit K_{BT} , describes the key material consumed by one physical-copy processing path. In actual execution with multiple physical copies, fresh key blocks with the same internal structure are drawn from the QKD key pool for each copy, and no QOTP key block is reused across different encrypted objects.

(I2) Trent generates a sufficient number of Bell states $|\varphi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. He sends one particle of a Bell state to Alice and the other to Bob, establishing a shared Bell state $|\varphi^+\rangle_{AB}$ for Step (S2). He sends one particle of another Bell state to Bob and keeps the other, establishing a shared Bell state for Step (S4). He sends one particle of another Bell state to Bob and keeps the other, establishing a shared Bell state for Step (S6). The exact numbers of Bell states used in these steps are given in the resource summary after the protocol description. We assume that these entanglement distribution steps can be

realized via standard quantum communication techniques, using quantum repeaters [27] if the parties are far away from each other [25]. The security of the distributed entanglement can be ensured by entanglement purification and quantum state authentication [28].

4.2.2. Signing-verifying phase

In this phase, a quantum message $|p\rangle$ is signed by Alice and verified by Trent and Bob. The first three steps are performed by Alice.

(S1) Alice prepares the quantum message in three groups: $|p_1\rangle$, $|p_2\rangle$, and $|p_3\rangle$. Each group consists of the required number of physical copies as specified in the resource summary. As discussed in Section 4.1, this is possible because Alice knows the classical description of $|p\rangle$. The notation $|p_1\rangle$, $|p_2\rangle$, $|p_3\rangle$ refers to these groups throughout the protocol.

(S2) Alice combines one physical copy of $|p_1\rangle$ with the Bell state $|\varphi^+\rangle_{AB}$ shared with Bob to obtain a three-particle entangled state, and performs a Bell measurement on the message particle and the Bell state particle in her hand. Then, the corresponding Bell state particle in Bob's hand becomes $U(X_1)|p_1\rangle$. Here, X_1 is determined by the measurement outcome in Alice's hand, and is determined randomly by the measurement outcome from the set $\{00, 01, 10, 11\}$. Note that Alice can obtain X_1 . In actual execution, this step is repeated for each physical copy of $|p_1\rangle$, and the corresponding measurement outcomes are denoted $X_1^{(i)}$.

(S3) Alice sends $|p_2\rangle$ and $E_{K'_{AT}}|p_3\rangle$ to Bob and Trent, respectively. After encrypting the second particle of the Bell state in her hand by $E_{K''_{AT}}$ and $|X_1\rangle$ by K'''_{AT} , she sends the particle pair (its state is $[I \otimes E_{K''_{AT}} U(X_1)]|\varphi^+\rangle$) and $E_{K'''_{AT}}|X_1\rangle$ to Trent. In actual execution, this step is repeated for each physical copy of $|p_1\rangle$ and the corresponding $X_1^{(i)}$ value.

Next step is performed by Bob.

(S4) Bob combines the received copy $U(X_1)|p_1\rangle$ with the Bell state shared with Trent (established in Step (I2) for this purpose) to obtain a three-particle entangled state, and performs a Bell measurement on the message particle and the Bell state particle in his hand. Then, the corresponding Bell state particle in Trent's hand becomes $U(X_2)U(X_1)|p_1\rangle$. Note that Bob can obtain X_2 . After encrypting the second particle of the Bell state in his hand by $E_{K'_{BT}}$ and $|X_2\rangle$ by K'''_{BT} , Bob sends the particle pair (its state is $[I \otimes E_{K'_{BT}} U(X_2)]|\varphi^+\rangle$) and $E_{K'''_{BT}}|X_2\rangle$ to Trent. In actual execution, this step is repeated for each received copy, and the corresponding measurement outcomes are denoted $X_2^{(i)}$.

The next two steps are performed by Trent.

(S5) Trent decrypts the particle pair from Alice using K'''_{AT} and measures it to obtain X_1 . He then decrypts $E_{K'''_{AT}}|X_1\rangle$ using K'''_{AT} to obtain $|X_1\rangle$, and compares the measured X_1 with the decrypted value. If they differ, the protocol aborts. Similarly, Trent decrypts the particle pair from Bob using K'_{BT} to obtain X_2 , and compares it with the decrypted $E_{K'_{BT}}|X_2\rangle$. If they differ, the protocol aborts. Trent then applies $U(X_1)$ and $U(X_2)$ to recover $|p_1\rangle$ from $U(X_2)U(X_1)|p_1\rangle$. In actual execution, this step is repeated for each copy, yielding $2\lambda + 1$ recovered copies $|p_1^{(i)}\rangle$. After all recovered copies and their corresponding protected records have arrived, Trent uses his own random generator to select λ recovered copies uniformly without replacement from the full set of $2\lambda + 1$ copies. He compares the selected copies one-to-one with the λ reference copies of $|p_3\rangle$ using independent swap tests as described in Section 4.1. If any test fails, the protocol aborts. Otherwise, the unselected $\lambda + 1$ copies of $|p_1\rangle$ proceed to Step (S6).

(S6) Trent combines one of the remaining copies of $|p_1\rangle$ with the Bell state shared with Bob (established in Step (I2) for this purpose) to obtain a three-particle entangled state, and performs a Bell measurement on the message particle and the Bell state particle in his hand. Then, the corresponding Bell state particle in Bob's hand becomes $U(X_3)|p_1\rangle$.

After encrypting the second particle of the Bell state in his hand by $E_{K_{BT}'''}^{(4)}$ and $|X_3\rangle$ by $K_{BT}^{(4)}$, Trent sends the particle pair (its state is $[I \otimes E_{K_{BT}'''}^{(4)} U(X_3)]|\varphi^+\rangle$) and $E_{K_{BT}^{(4)}}|X_3\rangle$ to Bob. In actual execution, this step is repeated for each remaining copy, and the corresponding measurement outcomes are denoted $X_3^{(i)}$.

Remark on the message flow. The signing-verifying phase realizes a two-stage teleportation chain that routes the message copy $|p_1\rangle$ along the closed path Alice $\rightarrow$ Bob $\rightarrow$ Trent $\rightarrow$ Bob. In the first stage, Alice teleports the state $U(X_1)|p_1\rangle$ to Bob (S2), and Bob teleports the received state to Trent (S4), so that after decrypting X_1 and X_2 , Trent can recover $|p_1\rangle$ without Alice and Trent sharing a direct teleportation channel. After Trent verifies the consistency between the teleported copy and the encrypted copy $|p_3\rangle$ (S5), the second stage begins: Trent teleports the verified message back to Bob (S6), who performs a final consistency check against his own copy $|p_2\rangle$ (S7). Each teleportation hop consumes one Bell pair and produces a two-bit classical outcome. The outcomes of the first two hops are denoted X_1 and X_2 , and the outcomes of the final hop are denoted X_3 . In actual execution, there are multiple such outcomes corresponding to the multiple physical copies, but the operation on each copy is identical to the single-copy description given here. All such outcomes are transmitted in encrypted form (X_1, X_2 to Trent, and X_3 to Bob). Except for the plaintext $|p_2\rangle$ reference copies transmitted from Alice to Bob, the protected quantum objects transmitted over insecure channels are encrypted using the strengthened QOTP. Consequently, Bob cannot forge the signature without being detected, because the teleported state he receives is encrypted by the unknown operation $U(X_1)$, and any tampering would be detected by Trent's swap test, and Trent cannot tamper with the message without invalidating Bob's final check. The overall transmission structure is illustrated in Figure 3.

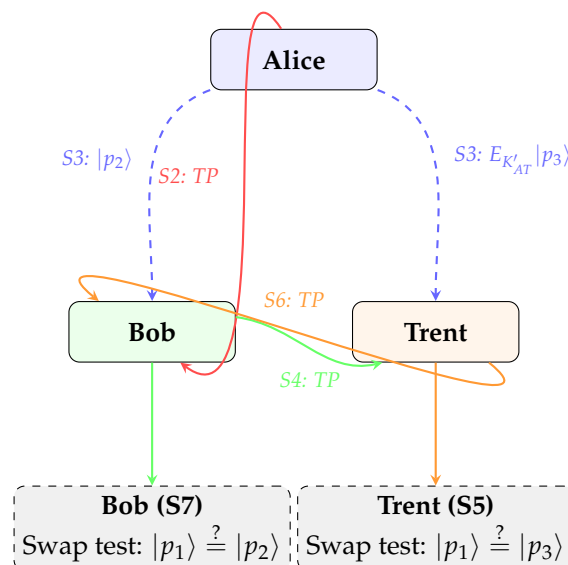


Figure 3. Transmission overview of the signing-verifying phase. Dashed straight arrows denote the direct distribution of the three message groups in S3. Curved arrows denote the teleportation hops S2, S4, and S6, which relay $|p_1\rangle$ along the path Alice $\rightarrow$ Bob $\rightarrow$ Trent $\rightarrow$ Bob.

The last step is performed by Bob to verify and accept message.

(S7) Bob decrypts the particle pair from Trent using K_{BT}''' and measures it to obtain X_3 . He then compares this value with the decrypted $|X_3\rangle$ value obtained by decrypting $E_{K_{BT}^{(4)}}|X_3\rangle$ using $K_{BT}^{(4)}$. If they differ, the protocol aborts. Otherwise, Bob applies $U(X_3)$ to recover one copy of $|p_1\rangle$. In actual execution, this step is repeated for each received copy, yielding $\lambda + 1$ recovered copies $|p_1^{(i)}\rangle$. After all returned copies and their corresponding protected records have arrived, Bob uses his own random generator to select λ recovered

copies uniformly without replacement from the full set of $\lambda + 1$ copies. He compares the selected copies one-to-one with the λ reference copies of $|p_2\rangle$ using independent swap tests as described in Section 4.1. If all tests accept, Bob accepts the batch and retains the unique unselected copy of $|p_1\rangle$ as the signed quantum message. Otherwise, the protocol aborts.

Resource summary. In actual execution, the numbers of physical copies and Bell pairs are as follows. Alice prepares $2\lambda + 1$ physical copies of $|p_1\rangle$, λ physical copies of $|p_2\rangle$, and λ physical copies of $|p_3\rangle$. Step (S2) consumes $2\lambda + 1$ Bell pairs shared between Alice and Bob. Step (S4) consumes $2\lambda + 1$ Bell pairs shared between Bob and Trent. Step (S5) consumes λ copies of $|p_1\rangle$ and λ copies of $|p_3\rangle$ in the swap test. Step (S6) consumes $\lambda + 1$ Bell pairs shared between Trent and Bob. Step (S7) consumes λ copies of $|p_1\rangle$ and λ copies of $|p_2\rangle$ in the swap test. After the protocol, Bob retains exactly one physical copy of $|p_1\rangle$ as the signed quantum message.

In summary, Alice transmits the quantum message to Trent via two rounds of teleportation through Bob. Trent verifies the message and teleports it back to Bob, who performs a final verification before acceptance. Except for the plaintext $|p_2\rangle$ reference copies transmitted from Alice to Bob, the protected quantum objects transmitted over insecure channels are encrypted using the strengthened QOTP, and all state comparisons are performed using the swap test. For clarity, the protocol execution flow is illustrated in Figure 4.

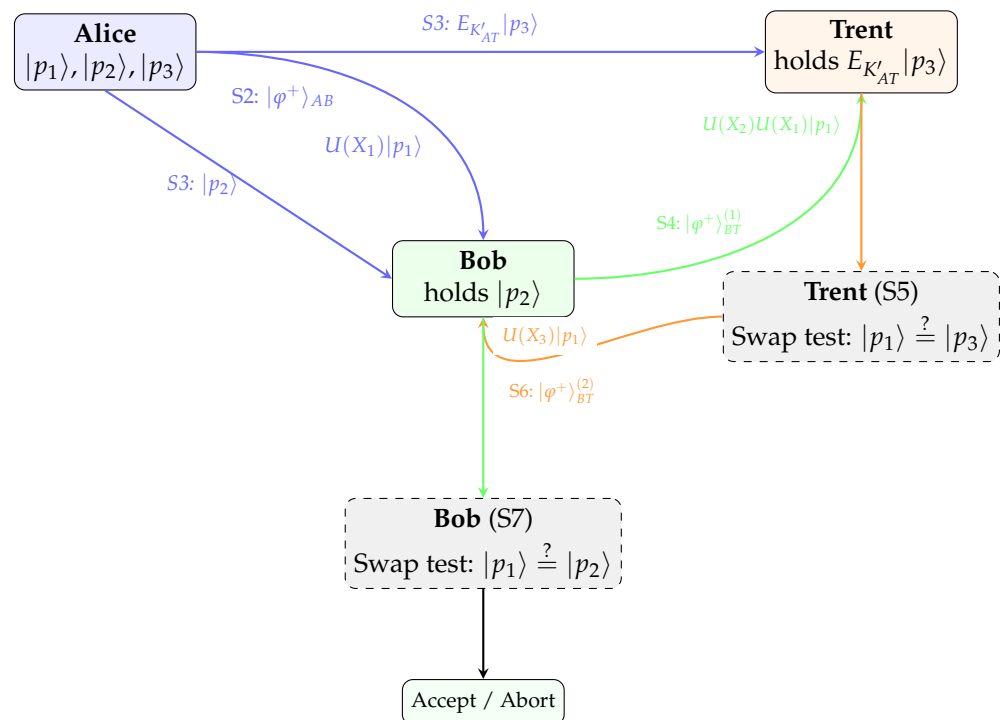


Figure 4. Flow of the three message groups and Bell state consumption in the proposed scheme.

4.3. Security analysis of the proposed scheme

We now analyze the security of the proposed scheme with respect to the formal definitions of non-repudiation and unforgeability established in Section 4.1. The analysis considers the three classes of adversaries defined in our model: a dishonest Alice, a dishonest Bob, and external adversaries. We then discuss the resilience of the protocol against other standard attack vectors.

4.3.1. Non-repudiation

We examine the proposed scheme with respect to the non-repudiation game defined in Section 4.1. Let $\mathcal{A}$ be a dishonest Alice who attempts to repudiate a valid signature.

In the signature generation phase of the game, Alice honestly produces a valid signature on a message $|p\rangle$ of her choice. The protocol executes normally with Bob and Trent following the specification. In the repudiation attempt phase, Alice outputs a claim that the signature is invalid and presents evidence to the arbitrator. For Alice to win, Trent must conclude that the signature is invalid despite Alice having genuinely generated it.

The analysis below considers a stronger adversary than the one defined in the non-repudiation game. Alice is allowed to deviate already during the signature generation phase, not only during the repudiation attempt phase. This extension is used to derive conditional fidelity-dependent consistency bounds; whether these bounds suffice for a complete proof of the stated non-repudiation game requires a separate theoretical check. Suppose Alice deviates from the protocol in every step where she has control. Specifically, in Step (S3), Alice sends $|p'_2\rangle$ to Bob, $E_{K'_{AT}}|p'_3\rangle$ to Trent, and a particle pair encoding X'_1 together with $E_{K'''_{AT}}|X'_1\rangle$ to Trent. These may differ arbitrarily from the honest values.

For the protocol to proceed beyond Step (S5), Trent must verify the submitted X_1 and X_2 records and all selected swap-test comparisons must accept. Assuming Bob behaves honestly, if Alice submits a value X'_1 , the state recovered by Trent on the corresponding physical path is

$$|p_1^{\text{rec}}\rangle = U(X'_1)U(X_1)|p_1\rangle \quad (6)$$

up to a global phase. This state equals the original $|p_1\rangle$ only when $X'_1 = X_1$, or when the combined operation acts equivalently on the particular input state. Consistency between Alice's particle-pair record and her encrypted X'_1 record does not by itself imply recovery of the original state.

For the j th comparison in Step (S5), let

$$F_{5,j} = \left| \langle p_3'^{(j)} | p_1^{\text{rec},(j)} \rangle \right|^2. \quad (7)$$

Conditional on the record checks passing, the probability that all λ independently selected S5 comparisons accept is

$$P_{\text{acc},5} = \prod_{j=1}^{\lambda} \frac{1 + F_{5,j}}{2}. \quad (8)$$

If every inconsistent tested pair satisfies the application-specific bound $F_{5,j} \leq F_{\max} < 1$, then $P_{\text{acc},5} \leq [(1 + F_{\max})/2]^\lambda$. A passing swap test is therefore a probabilistic acceptance event and does not logically imply exact equality of the compared states.

Likewise, for the selected comparisons in Step (S7), define $F_{7,j}$ as the fidelity between the returned recovered copy and Bob's corresponding $|p'_2\rangle$ reference copy. Conditional on reaching Step (S7), the probability that all selected comparisons accept is

$$P_{\text{acc},7} = \prod_{j=1}^{\lambda} \frac{1 + F_{7,j}}{2}. \quad (9)$$

Thus, Bob's acceptance provides fidelity-dependent statistical evidence that the selected pairs passed the verification tests; it does not by itself imply the deterministic relation $|p'_2\rangle \cong |p_1\rangle \cong |p'_3\rangle$.

These bounds quantify the false-acceptance probability of the tested comparisons. A complete non-repudiation argument must additionally establish, under the stated adversarial game, how the authenticated records and the verifier-chosen random sampling bind Alice to the accepted batch, including the unique unselected output copy. The numerical consistency test alone does not establish this protocol-level property.

4.3.2. Unforgeability

We show that the proposed scheme satisfies the unforgeability game defined in Section 4.1. Let $\mathcal{A}$ be a polynomial-time adversary, which may be either a dishonest Bob or an external adversary. We consider each case separately.

Dishonest Bob. A dishonest Bob possesses the key K_{BT} and participates in the protocol as the designated receiver. His goal is to forge a valid signature—that is, to produce a message-signature pair that passes Trent’s verification and is attributed to Alice, without Alice having signed that message.

Bob’s capabilities in the protocol are as follows. After Step (S3), Bob holds $|p_2\rangle$ (the plaintext copy sent by Alice) and $U(X_1)|p_1\rangle$ (the teleported state). In Step (S4), Bob performs a Bell measurement and learns X_2 . After Step (S7), Bob additionally holds $U(X_3)|p_1\rangle$ (the state teleported back by Trent) and learns X_3 .

To forge a signature, Bob must produce a message $|p^*\rangle$ and evidence that causes Trent to attribute it to Alice. There are two stages at which Bob might attempt to cheat.

In the first stage (during protocol execution), Bob could modify $|p_2\rangle$ or apply a non-identity operation G on $U(X_1)|p_1\rangle$ before teleporting it to Trent in Step (S4). If Bob applies G , the state reaching Trent in Step (S5) becomes $U(X_2)GU(X_1)|p_1\rangle$. For the protocol to proceed, Trent must successfully verify X_1 and X_2 . The verification of X_1 depends on Alice’s encryption with K_{AT} . By the information-theoretic security of the QOTP, Bob cannot determine X_1 from the ciphertext without K_{AT} . If Bob sends an inconsistent X_2 , the decryption check in Step (S5) fails and the protocol aborts. If Bob passes the X_2 check but modified the quantum state, Trent applies $U(X_1)$ and $U(X_2)$ to the received state. Since Pauli operators either commute or anti-commute, the recovered state becomes $U(X_1)GU(X_1)|p_1\rangle$ (up to a global phase). Trent then compares it with $|p_3\rangle$ via the swap test. Let $F_G = |\langle p_1 | U(X_1)GU(X_1) | p_1 \rangle|^2$ denote the fidelity between the recovered state and the reference state $|p_3\rangle$. The swap test detects the mismatch with probability $1 - [(1 + F_G)/2]^\lambda$. When $G = I$, we have $F_G = 1$ and the test always accepts. When $G \neq I$, the detection probability depends on F_G ; in the worst case for the protocol, G may be chosen such that $F_G \approx 1$ (e.g., a near-identity operation that preserves the message up to a small perturbation), yielding a low detection probability. We now reduce Bob’s forgery success to the security of the underlying primitives. Bob holds the plaintext copy $|p_2\rangle = |p_1\rangle$, so he can in principle produce a state with fidelity $F_G = 1$ to the reference. However, such a state is exactly the original message signed by Alice, and presenting it as a forgery yields no advantage: it is the message Alice already signed. To forge a different message, Bob must produce a state $|p^*\rangle \neq |p_1\rangle$, whose fidelity to the reference satisfies $F_G < 1$. The swap test then detects the mismatch with probability $1 - [(1 + F_G)/2]^\lambda$. Bob cannot make F_G arbitrarily close to 1 while keeping $|p^*\rangle$ semantically different from $|p_1\rangle$: under the fidelity threshold $F_{\max}$, any semantically meaningful forgery satisfies $F_G \leq F_{\max}$, so the detection probability is at least $1 - [(1 + F_{\max})/2]^\lambda$. Therefore, Bob’s forgery success probability is at most $[(1 + F_{\max})/2]^\lambda$, which is negligible in λ .

In the second stage (after protocol completion), Bob possesses $|p_2\rangle$ and $U(X_3)|p_1\rangle$. After applying $U(X_3)$, Bob recovers $|p_1\rangle$ and compares it with $|p_2\rangle$. At this point, Bob already holds the valid message. To forge a different message, Bob would need to produce a new $|p^*\rangle$ and corresponding evidence that Trent accepts as originating from Alice. However, Trent retains the authenticated classical transcript of the protocol execution, including the record of Alice’s encrypted transmission, the classical parameters X_1 and X_2 , and the swap-test outcomes. Any forgery attempt by Bob would require him to produce evidence consistent with this authenticated transcript. Since the transcript is authenticated under keys that Bob does not possess, Bob cannot alter it without detection.

Bob's knowledge of X_2 and X_3 does not assist in forgery, because these values are random Bell measurement outcomes that reveal no information about the signed message. They are independent of the message content and are used solely for reconstructing the teleported state.

Therefore, a dishonest Bob cannot win the unforgeability game with non-negligible probability.

External adversary. An external adversary does not possess any of the pre-shared keys K_{AT} or K_{BT} . Except for the plaintext $|p_2\rangle$ reference copies transmitted from Alice to Bob, the protected quantum objects transmitted over insecure channels are encrypted using the strengthened QOTP, which provides information-theoretic confidentiality when the encryption keys are uniformly random and used only once. Specifically, in Step (S3), Alice encrypts $|p_3\rangle$ with K'_{AT} and the Bell state particle pair with K''_{AT} . In Step (S4), Bob encrypts his particle pair with K'_{BT} . In Step (S6), Trent encrypts his particle pair with K'''_{BT} . Without access to these keys, the adversary cannot decrypt the ciphertext to recover the underlying quantum states, nor can it extract meaningful information via the attacks discussed in Section 4.3.3. Consequently, the adversary cannot extract the plaintext from the ciphertext. Furthermore, any active interference by the adversary, such as intercepting and replacing a quantum state in transit, introduces discrepancies that are detected by the subsequent consistency checks, not by the QOTP encryption itself. These discrepancies manifest in two ways. First, the decrypted values X_1 , X_2 , and X_3 obtained by Trent and Bob will not match the encrypted values sent by the legitimate parties, causing the protocol to abort in Steps (S5) and (S7). Second, even if the adversary somehow managed to preserve the classical parameters, the swap test comparisons in Steps (S5) and (S7) would detect the mismatch between the recovered $|p_1\rangle$ and the stored copies $|p_3\rangle$ and $|p_2\rangle$, respectively. The swap test fails to detect a difference with probability $[(1 + F)/2]^\lambda \leq [(1 + F_{\max})/2]^\lambda$, where F is the fidelity between the recovered state and the reference state, and $F_{\max}$ is the fidelity threshold defined in Section 4.1. For orthogonal states ($F = 0$), this reduces to $2^{-\lambda}$, which is negligible. We now summarize the external adversary's success probability. To win the unforgeability game, the adversary must produce a message-signature pair that passes Trent's verification. QOTP confidentiality reveals no information about the plaintext beyond the adversary's prior knowledge; it does not make the probability of guessing a plaintext exactly zero. Consequently, unforgeability must be established from the complete authenticated verification conditions rather than from QOTP confidentiality alone. Under an explicitly stated message prior and attack model, any residual success bound must account for both the adversary's prior guessing probability and the fidelity-dependent probability of passing the complete verification procedure.

4.3.3. Security against other attacks

We now discuss the resilience of the proposed scheme against several additional types of attacks that are relevant to quantum signature protocols.

Replay attacks. A replay attack against the proposed scheme would consist of recording the quantum states and classical messages transmitted during a legitimate execution and later retransmitting them in a new session. Two protocol features prevent this. First, every execution uses freshly generated QKD keys K_{AT} and K_{BT} . The replayed states, encrypted under an old key, would not decrypt correctly under the current key held by Trent or Bob, causing the verification in Steps (S5) or (S7) to fail. Second, the Bell measurement outcomes X_1 , X_2 , and X_3 are independently random in each execution. Even if a replayed quantum state were somehow accepted, the accompanying classical X_i values from the previous session would not match the values independently measured by Trent and Bob in the current session, and the comparisons in Steps (S5) and (S7) would abort the protocol.

Replay attacks are therefore prevented by fresh per-execution QKD keys and independently random Bell outcomes.

Intercept-and-resend attacks. In the proposed scheme, an adversary who intercepts a quantum transmission, for instance the particle pair sent by Alice to Trent in Step (S3), faces two obstacles. First, the state is encrypted via the strengthened QOTP using K''_{AT} , which the adversary does not possess. The QOTP renders the ciphertext statistically indistinguishable from a maximally mixed state, so any measurement yields no information about the underlying plaintext. Second, if the adversary replaces the intercepted state with an arbitrary state and forwards it to Trent, the decryption in Step (S5) will produce a value that does not match the separately transmitted encrypted record $E_{K''_{AT}}|X_1\rangle$, causing the protocol to abort. The same argument applies to the particle pairs transmitted in Steps (S4) and (S6). If the adversary instead targets the plaintext copy $|p_2\rangle$ or the encrypted copy of $|p_3\rangle$, any modification will be detected by the swap test comparisons in Steps (S5) and (S7) with probability at least $1 - [(1 + F_{\max})/2]^\lambda$. We note that QOTP provides confidentiality and plaintext hiding; integrity and authentication are enforced by these additional protocol checks. The adversary therefore gains no advantage from intercept-and-resend attacks.

Man-in-the-middle attacks. A man-in-the-middle adversary must simultaneously impersonate one party to another on both directions of a communication channel. In the proposed scheme, this requires overcoming three independent defenses. First, the classical channels are authenticated, so any modification of the encrypted X_i values or other classical messages is detected. Second, except for the plaintext $|p_2\rangle$ reference copies transmitted from Alice to Bob, the protected quantum objects transmitted over insecure channels are encrypted using the strengthened QOTP; without the corresponding key, the adversary obtains no plaintext information from those ciphertexts. Third, even if the adversary successfully impersonates one party on one leg, the cross-verification performed by Trent in Step (S5) compares the independently received X_1 and X_2 values from Alice and Bob. A discrepancy introduced on any single channel, for example by replacing the particle pair and encrypted X_1 sent by Alice to Trent in Step (S3), will be detected when Trent decrypts and finds mismatching values. The adversary cannot replace X_1 without being detected, because the classical authentication check would fail. Symmetric arguments apply to the other communication legs. Under the assumptions of authenticated classical channels and confidential pre-shared keys, man-in-the-middle attacks are effectively prevented.

Entanglement attacks. An adversary may attempt to entangle an ancilla with a transmitted quantum state and later measure the ancilla to extract information. Except for the plaintext $|p_2\rangle$ reference copies transmitted from Alice to Bob, the protected quantum objects transmitted over insecure channels are encrypted using the strengthened QOTP. The ciphertext appears maximally mixed to any party without the decryption key. If the adversary entangles an ancilla with the ciphertext, the reduced state of the ancilla remains independent of the plaintext. Formally, for a QOTP-encrypted state $\frac{1}{2^{2n}} \sum_{a,b} X^a Z^b \rho Z^b X^a \otimes |a,b\rangle\langle a,b|$, tracing out the ciphertext leaves the ancilla uncorrelated with the original message ρ . The adversary therefore gains no information, and this protection holds against computationally unbounded adversaries due to the information-theoretic security of the QOTP.

Collective and coherent attacks. In collective attacks, the adversary processes each quantum signal individually but stores ancillae for joint measurement across multiple protocol runs. In coherent attacks, all signals may be processed jointly. The proposed scheme resists these attacks in the sense of preserving plaintext confidentiality across executions. Each execution uses independent, freshly generated QKD keys, so the encrypted states from different runs are statistically independent. Joint measurement across multiple ciphertexts yields no more information than individual analysis. Within a single execution,

the security follows from the standard QOTP confidentiality guarantee, which holds against unbounded adversaries. The keys themselves are generated by QKD and are information-theoretically secure.

As above, we have shown that the proposed scheme satisfies non-repudiation and unforgeability under the formal adversarial model defined in Section 4.1. The analysis considers dishonest internal parties and external adversaries separately, and shows that neither class can succeed with non-negligible probability. Together, QOTP encryption, authenticated classical channels, cross-verification by Trent, and per-execution fresh keys protect the protocol against replay, intercept-and-resend, man-in-the-middle, entanglement, and collective or coherent attacks.

5. Resource overhead and simulation

This section first analyzes resource overheads of the proposed scheme and YKLY'2015, and then simulates the attacks on the proposed scheme.

5.1. Resource overhead analysis of the proposed scheme and YKLY'2015

To assess the practical cost of the security guarantees established in Section 4.3, we now analyze the resource consumption of the proposed scheme. For a single quantum message, all core operations of the proposed scheme maintain constant-level computational complexity. In this subsection, we divide the protocol execution into the initialization stage (I1-I2) and the signal interaction stage (S1-S7) to conduct a comprehensive resource overhead analysis, covering key consumption, qubit overhead, quantum communication overhead, classical communication overhead, and computational complexity of each step.

Step (I1): This step generates and allocates a sufficiently large QKD key pool for the complete physical-copy batch. The 16-bit K_{AT} and 24-bit K_{BT} partitions describe the key-block pattern for one physical processing path: 4 bits for each protected message or Bell-particle object and 8 bits for each protected Bell-outcome record. In an actual execution, every encrypted physical object receives an independently allocated, single-use key block with its own block identifier. For an n -qubit message, the required fresh key material is $(64\lambda + 36)n$ bits in total. This quantity is allocated in Step (I1) and consumed by the protected objects created in Steps (S3), (S4), and (S6); the allocation and later consumption are not counted twice. The key-generation and allocation cost therefore scales as $O(\lambda n)$.

Step (I2): No additional secret keys are consumed in this step. $5\lambda + 3$ Bell states are prepared and serve as persistent entanglement resources, occupying $2(5\lambda + 3) = 10\lambda + 6$ static qubits. Among them, $2\lambda + 1$ Bell states are shared between Alice and Bob for Step (S2), $2\lambda + 1$ Bell states are shared between Bob and Trent for Step (S4), and $\lambda + 1$ Bell states are shared between Trent and Bob for Step (S6). A total of $7\lambda + 4$ qubits are transmitted from Trent to Alice and Bob during entanglement distribution. For the $2\lambda + 1$ Alice–Bob Bell pairs, both particles leave Trent, giving $2(2\lambda + 1) = 4\lambda + 2$ transmitted qubits. For the $2\lambda + 1$ Bob–Trent Bell pairs used in Step (S4), one particle is sent to Bob while the other is retained by Trent, giving $2\lambda + 1$ transmitted qubits. For the $\lambda + 1$ Bob–Trent Bell pairs used in Step (S6), one particle is sent to Bob while the other is retained by Trent, giving $\lambda + 1$ transmitted qubits. These Bell states must be preserved until their respective teleportation steps. Let t_{comm} denote the one-way classical communication latency between any two parties, and let t_{op} denote the time for local quantum operations and measurements. Under a fully serial schedule for all physical-copy teleportations, the total protocol execution time, which determines the required storage duration, is $t_{\text{total}} = (5\lambda + 3)t_{\text{comm}} + O(t_{\text{op}})$. This latency expression corresponds to serial processing of the physical copies; parallel or pipelined execution can reduce the number of sequential communication intervals. For metropolitan-scale distances on the order of tens of kilometers, the serial estimate places

t_{total} on the order of milliseconds when t_{comm} is on the order of tens of microseconds in optical fiber. This is well within the coherence times demonstrated by current quantum memory technologies. For longer-distance applications, the required storage time increases accordingly, and practical deployment would require continued progress in coherence-preserving technologies.

Step (S1): No key resources are consumed. $4\lambda + 1$ dynamic qubits are occupied for quantum message storage, realized as $2\lambda + 1$ copies of $|p_1\rangle$, λ copies of $|p_2\rangle$, and λ copies of $|p_3\rangle$. There is no quantum or classical communication transmission overhead. The operation includes single-qubit state preparation for all physical copies, with a computational complexity of $O(\lambda)$.

Step (S2): In actual execution, this step is repeated for $2\lambda + 1$ copies of $|p_1\rangle$, consuming $2\lambda + 1$ Bell pairs shared between Alice and Bob, and performing $2\lambda + 1$ Bell measurements. No classical bits are transmitted in this step. The computational complexity is $O(\lambda)$.

Step (S3): For an n -qubit message, this step consumes $(28\lambda + 12)n$ fresh key bits: $4\lambda n$ bits for the protected $|p_3\rangle$ copies and $(4 + 8)(2\lambda + 1)n$ bits for the protected S3 Bell-particle objects and X_1 records. No new persistent qubits are added. Alice sends λ copies of $|p_2\rangle$ to Bob and λ copies of $E_{K'_{AT}}|p_3\rangle$ to Trent. In addition, for each of the $2\lambda + 1$ copies of $|p_1\rangle$, Alice sends one encrypted particle pair, consisting of two qubits, to Trent, giving $2(2\lambda + 1) = 4\lambda + 2$ additional qubits. The total quantum communication in this step is therefore $\lambda + \lambda + 4\lambda + 2 + 4\lambda + 2 = 10\lambda + 4$ qubits.

Step (S4): For an n -qubit message, this step consumes $(24\lambda + 12)n$ fresh key bits for the protected Bell-particle objects and X_2 records on the $2\lambda + 1$ received physical paths. In actual execution, this step is repeated for $2\lambda + 1$ received copies. A total of $2(2\lambda + 1) = 4\lambda + 2$ encrypted Bell particle qubits are transmitted, accompanied by $2\lambda + 1$ encrypted parameters $E_{K''_{BT}}|X_2^{(i)}\rangle$, each transmitted as a two-qubit quantum state, adding $2(2\lambda + 1) = 4\lambda + 2$ qubits. The total quantum communication in this step is therefore $8\lambda + 4$ qubits. The core operations are Bell measurement and QOTP encryption, and the computational complexity is $O(\lambda)$.

Step (S5): No new key segments and persistent qubits are consumed. There is no quantum and classical communication overhead. In actual execution, Trent recovers $2\lambda + 1$ copies of $|p_1\rangle$. Among them, λ copies are used in the swap test with $|p_3\rangle$, and the remaining $\lambda + 1$ copies are retained for Step (S6). The main operations are quantum state recovery and λ independent swap tests, with a computational complexity of $O(\lambda)$.

Step (S6): For an n -qubit message, this step consumes $(12\lambda + 12)n$ fresh key bits for the protected Bell-particle objects and X_3 records on the $\lambda + 1$ returned physical paths. No extra persistent qubits are introduced. A total of $2(\lambda + 1) = 2\lambda + 2$ encrypted Bell particle qubits are transmitted, accompanied by $\lambda + 1$ encrypted parameters $E_{K^{(4)}_{BT}}|X_3^{(j)}\rangle$, each transmitted as a two-qubit quantum state, adding $2(\lambda + 1) = 2\lambda + 2$ qubits. The total quantum communication in this step is therefore $4\lambda + 4$ qubits. The step integrates Bell measurement and QOTP encryption, with an overall computational complexity of $O(\lambda)$.

Step (S7): No key resources and persistent qubits are consumed, with zero communication overhead of quantum and classical channels. In actual execution, Bob receives $\lambda + 1$ copies of $|p_1\rangle$. He uses λ copies in the swap test with $|p_2\rangle$ and retains the remaining one copy as the signed quantum message. The core operations are Pauli correction and λ independent swap tests, and the computational complexity is $O(\lambda)$.

As above, the stepwise resource overhead summary of the proposed scheme is presented in Table 2. By the same approach, we derive the resource overhead of the Grover-based scheme (i.e., YKLY'2015), as listed in Table 3. The proposed scheme differs architecturally from YKLY'2015 in several respects. The proposed scheme introduces persistent static qubits and consumes more key material ($O(\lambda)$ bits per execution vs. 4 bits in

Table 2. Stepwise resource overhead summary of the proposed scheme

Index	I1	I2	S1	S2	S3	S4	S5	S6	S7	Total
Fresh Key Allocation/Consumption (bit)	$(64\lambda + 36)n$ allocated	0	0	0	$(28\lambda + 12)n$	$(24\lambda + 12)n$	0	$(12\lambda + 12)n$	0	$(64\lambda + 36)n$ consumed
Static and Dynamic Qubits	0	$2(5\lambda + 3)$	$4\lambda + 1$	0	0	0	0	0	0	$14\lambda + 7$
Quantum Transmit (qubit)	0	$7\lambda + 4$	0	0	$10\lambda + 4$	$8\lambda + 4$	0	$4\lambda + 4$	0	$29\lambda + 16$
Classical Transmit (bit)	$O(\lambda)$	0	0	0	0	0	0	0	0	$O(\lambda)$
Computational Complexity	$O(\lambda)$	$O(\lambda)$	$O(\lambda)$	$O(\lambda)$	$O(\lambda)$	$O(\lambda)$	$O(\lambda)$	$O(\lambda)$	$O(\lambda)$	$O(\lambda)$

¹ Step (I1) allocates $(64\lambda + 36)n$ fresh QKD key bits for the complete batch. The S3, S4, and S6 entries show where those allocated bits are consumed; allocation and consumption are not added together. Each encrypted physical object receives a distinct single-use block identifier, although independently sampled blocks may coincidentally contain the same bit pattern.

² Maximum number of simultaneously occupied valid qubits throughout a complete protocol execution cycle (values are non-cumulative).

³ Quantum/Classical Transmission: Cumulative qubits/classical bits transmitted over all protocol stages, including communication for QKD initialization.

⁴ Computational Complexity: Asymptotic complexity for all quantum and classical operations within each step.

Table 3. Stepwise resource overhead summary of YKLY'2015

Index	AT Total (AT1-AT8)	TB Total (TB1-TB8)	Overall Total
Key Consumed (bit)	2	2	4
Dynamic Qubits Used	8	8	8
Quantum Transmit (qubit)	8	8	16
Classical Transmit (bit)	13	13	26
Computational Complexity	$O(1)$	$O(1)$	$O(1)$

¹ Key Consumption: New secret key bits generated in each stage; the total row accumulates all independent QKD keys K_{AT} and K_{TB} .

² Dynamic Qubit Consumption: Temporary qubits occupied within each step (values are non-cumulative).

³ Quantum/Classical Transmission: Cumulative transmitted qubits and classical bits within each step.

⁴ Computational Complexity: Asymptotic complexity of all quantum and classical operations within each step.

YKLY'2015). These additional costs are incurred in exchange for stronger security guarantees. Unlike YKLY'2015, which is vulnerable to man-in-the-middle attacks that enable Alice's disavowal and Bob's forgery, the proposed scheme achieves unforgeability and non-repudiation through the combined use of strengthened QOTP encryption and arbitrated teleportation-based verification. The higher key consumption reflects the additional encryption and authentication steps required to protect against these attacks. Furthermore, quantum teleportation is adopted to reuse quantum channels, reducing the need for repeated quantum state transmissions.

The above analysis considers a single-qubit message. For a general n -qubit message $|p\rangle$, the resource requirements scale as follows.

Key consumption. Each qubit of the message requires independent encryption via the strengthened QOTP, which consumes a constant number of key bits per qubit. Specifically, for an n -qubit message, Alice must encrypt $4\lambda + 1$ physical copies of the message and the Bell state particle pairs used in teleportation. The X_i parameters each become $2n$ -bit classical values for an n -qubit message. There are $5\lambda + 3$ such parameters in total: $2\lambda + 1$ values of X_1 , $2\lambda + 1$ values of X_2 , and $\lambda + 1$ values of X_3 . Their encryption cost therefore scales as $O(\lambda n)$. The total key consumption therefore scales as $O(\lambda n)$, with a dominant contribution coming from the QOTP encryption of the n -qubit message copies.

Static qubits. Each teleportation of an n -qubit message consumes n Bell states. The protocol involves $5\lambda + 3$ rounds of teleportation, requiring $(5\lambda + 3)n$ Bell states in total. Each Bell state occupies 2 qubits, yielding $2(5\lambda + 3)n$ static qubits. The total number of static qubits therefore scales as $O(\lambda n)$.

Dynamic qubits. Alice prepares $4\lambda + 1$ physical copies of the n -qubit message, occupying $(4\lambda + 1)n$ dynamic qubits. During teleportation, each message qubit temporarily

interacts with one half of a Bell state, but no additional persistent storage is required beyond the message copies themselves. The total dynamic qubit requirement is therefore $O(\lambda n)$.

Quantum communication. In Step (S3), Alice transmits the message group $|p_2\rangle$ (λn qubits) to Bob and the encrypted message group $E_{K'_{AT}}|p_3\rangle$ (λn qubits) to Trent. In addition, the Bell state particle pairs transmitted in Steps (S3), (S4), and (S6) scale as $O(\lambda n)$ in total, since these steps involve $2\lambda + 1$, $2\lambda + 1$, and $\lambda + 1$ teleportations, respectively. The total quantum communication overhead is therefore $O(\lambda n)$, dominated by the transmission of the message groups.

Classical communication. The QKD initialization step (I1) requires $O(\lambda n)$ classical bits for key distribution, as the total key length scales with λn . All encrypted X_i parameters are transmitted as quantum states and are therefore counted in the quantum communication overhead rather than the classical communication overhead. The total classical communication overhead is therefore $O(\lambda n)$, dominated by QKD key distribution.

Computational complexity. The dominant operations are Bell measurements, QOTP encryption and decryption, and swap tests. For an n -qubit message, Bell measurements are performed qubit-wise on each of the $5\lambda + 3$ teleportation rounds, requiring $O(\lambda n)$ operations. The QOTP encryption applies Pauli gates independently to each qubit of the $4\lambda + 1$ physical copies, scaling as $O(\lambda n)$. Each swap test compares n -qubit registers using n controlled-SWAP gates, and the test is repeated λ times per comparison. Two comparisons are performed (in Steps S5 and S7), yielding $O(\lambda n)$ overall. The total computational complexity is therefore $O(\lambda n)$.

In summary, for an n -qubit message, key consumption, qubit overhead, communication requirements, and computational complexity all scale as $O(\lambda n)$. This scaling remains linear in the message length n , which is a natural consequence of processing each qubit individually. Table 4 summarizes the asymptotic resource scaling. Since λ is fixed once the security level and the fidelity threshold $F_{\max}$ are chosen, all resource requirements remain linear in the message length n .

Table 4. Asymptotic resource scaling for an n -qubit message.

Resource	Scaling
Key consumption	$O(\lambda n)$
Static qubits	$O(\lambda n)$
Dynamic qubits	$O(\lambda n)$
Quantum communication	$O(\lambda n)$
Classical communication	$O(\lambda n)$
Computational complexity	$O(\lambda n)$

5.2. Attack simulations of the proposed scheme

To complement the formal security analysis of Section 4.3, we now analyze the strongest attacks that a dishonest Alice or Bob can mount against the proposed scheme, and illustrate their behavior through numerical modeling. By contrast, the vulnerabilities of YKLY'2015 follow directly from its protocol structure and require no simulation to verify.

5.2.1. Alice's attack simulation

For Alice's repudiation, we construct the strongest possible attack, in which Alice deviates maximally in every step under her control.

Alice's attack description. In Step (S3), Alice deviates from the protocol by sending a potentially forged state $|p'_2\rangle$ to Bob and $E_{K'_{AT}}|p'_3\rangle$ to Trent, where $|p'_2\rangle$ and $|p'_3\rangle$ may differ from the honest copies. Simultaneously, she sends a forged particle pair with state $[I \otimes E_{K''_{AT}} U(X'_1)]|\varphi^+\rangle$ and $E_{K'''_{AT}}|X'_1\rangle$ to Trent, where X'_1 may differ from the true X_1 .

Verification phase Analysis. In Step (S5), Trent decrypts and verifies the received particles. We analyze three possible scenarios depending on Bob's behavior, since Alice does not control whether Bob is honest.

Scenario 1: Bob is honest. Bob follows the protocol ($G = I$, $X'_2 = X_2$). Trent measures X'_1 from Alice's particle pair and compares it with the decrypted $|X'_1\rangle$. If they match, Trent applies $U(X'_1)$ and $U(X_2)$ to the state $U(X_2)U(X_1)|p_1\rangle$. The recovered state is $|p_1^{\text{rec}}\rangle = U(X'_1)U(X_1)|p_1\rangle$ (up to a global phase). The swap test between $|p_1^{\text{rec}}\rangle$ and $|p'_3\rangle$ yields the condition

$$|p_1^{\text{rec}}\rangle \cong |p'_3\rangle \cong U(X'_1)U(X_1)|p_1\rangle. \quad (10)$$

This corresponds to the stronger malicious-signing adversary considered in the non-repudiation analysis of Section 4.3, where Alice is allowed to deviate during signature generation.

Scenario 2: Bob is dishonest, but his attack may prevent the protocol from proceeding. Bob applies $G \neq I$ or sends $X'_2 \neq X_2$. A non-identity effective operation does not by itself imply $F < 1$. Detection depends on the state-level fidelity F between the state recovered by Trent and the reference state $|p_3\rangle$; if the effective operation preserves the tested state up to a global phase, then $F = 1$ and the Swap-Test component does not detect it. When $F < 1$, the detection probability for λ independent comparisons is $1 - [(1 + F)/2]^\lambda$, and under an explicitly imposed bound $F \leq F_{\text{max}} < 1$ it is at least $1 - [(1 + F_{\text{max}})/2]^\lambda$. If the protocol aborts in this scenario, no signature has been accepted for Alice to repudiate.

Scenario 3: Bob is dishonest, but his attack inadvertently assists Alice. Bob applies $G \neq I$ and/or sends $X'_2 \neq X_2$, but the combination happens to satisfy $U(X'_2)U(X_2)G = I$. In this case, the swap test passes and the protocol proceeds despite Bob's deviation. Table 5 shows the required G for each pair (X'_2, X_2) . For a given pair (X'_2, X_2) , Bob's deviation G is chosen randomly from $\{I, \sigma_x, \sigma_y, \sigma_z\}$, and exactly one of these four choices satisfies $U(X'_2)U(X_2)G = I$. Hence, if Bob chooses G uniformly at random, the probability that his deviation inadvertently allows the protocol to proceed is $1/4$. Even in this scenario, once the protocol proceeds to Step (S7) and Bob accepts, the traceability argument of Scenario 1 applies, and Trent can still link the accepted message to Alice.

Table 5. Required operation G for the protocol to proceed when Bob deviates by sending X'_2 instead of the true X_2 . The diagonal entries ($X'_2 = X_2$) correspond to $G = I$, i.e., Bob does not apply any additional operation.

$X'_2 \backslash X_2$	00	01	10	11
00	I	σ_x	σ_z	σ_y
01	σ_x	I	σ_y	σ_z
10	σ_z	σ_y	I	σ_x
11	σ_y	σ_z	σ_x	I

In practical systems, non-attack factors such as channel noise, device imperfections, and environmental disturbances introduce an additional baseline error rate, denoted by α . In Scenario 1 (Bob honest), the protocol proceeds deterministically from Alice's perspective, and the only source of error is the finite failure probability $[(1 + F)/2]^\lambda$ of the swap test, which is bounded by $[(1 + F_{\text{max}})/2]^\lambda$ and reduces to $2^{-\lambda}$ when $F_{\text{max}} = 0$. In Scenario 3 (Bob inadvertently assists Alice), the protocol proceeds with ideal probability $1/4$ as shown above. Taking the worst-case view that Scenario 3 occurs, the total probability that Trent observes an erroneous acceptance is modeled as $p = \frac{1}{4} + \alpha$, where the $1/4$ term accounts for the probability that Bob's random deviation accidentally satisfies $U(X'_2)U(X_2)G = I$, and α is the baseline error rate due to physical noise, treated as an independent additive contribution in this phenomenological model. A detection threshold β is set: the transmission is accepted if the observed bit-error rate $\hat{\epsilon} \leq \beta$, and rejected otherwise. The probability

that Alice's attack succeeds (remains undetected) is $P_{\text{success}}(n) = \Pr(\hat{\varepsilon} \leq \beta \mid p = \frac{1}{4} + \alpha)$. For n transmitted quantum bits, let the observed number of erroneous bits be k . Then k follows a binomial distribution $k \sim \text{Binomial}(n, p)$, and the observed bit-error rate is $\hat{\varepsilon} = k/n$. Hence,

$$P_{\text{success}}(n) = \sum_{k=0}^{\lfloor n\beta \rfloor} \binom{n}{k} \left(\frac{1}{4} + \alpha\right)^k \left(\frac{3}{4} - \alpha\right)^{n-k}. \quad (11)$$

The security requirements are $p > \beta$. The condition $p > \beta$ ensures that the bit-error rate induced by Alice's attack exceeds the detection threshold, meaning the attack leaves a detectable trace and is therefore likely to be rejected by the system. Consequently, the attack success probability $P_{\text{success}}(n)$ represents the probability that the system accepts the transmission despite an actual attack being present, i.e., the false negative rate. By Chernoff's bound for the binomial distribution, for $\beta < 1/4 + \alpha$,

$$P_{\text{success}}(n) \leq \exp \left[-n \left(\beta \ln \frac{\beta}{1/4 + \alpha} + (1 - \beta) \ln \frac{1 - \beta}{3/4 - \alpha} \right) \right]. \quad (12)$$

For clarity, the upper bound on Alice's attack success probability as a function of the number of quantum bits n is shown in Fig. 5. We adopt the Monte-Carlo method to conduct numerical simulations. For each value of qubit number n , we perform 20000 random sampling experiments obeying binomial distribution $k \sim \text{Binomial}(n, 0.30)$. We judge that the attack succeeds when $k \leq n\beta$ with $\beta = 0.06$. It can be seen that the simulation results fit well with the theoretical binomial expression, which confirms the consistency of the numerical implementation with the probabilistic model. The MATLAB code implementing the Monte-Carlo simulation is given in Appendix A.

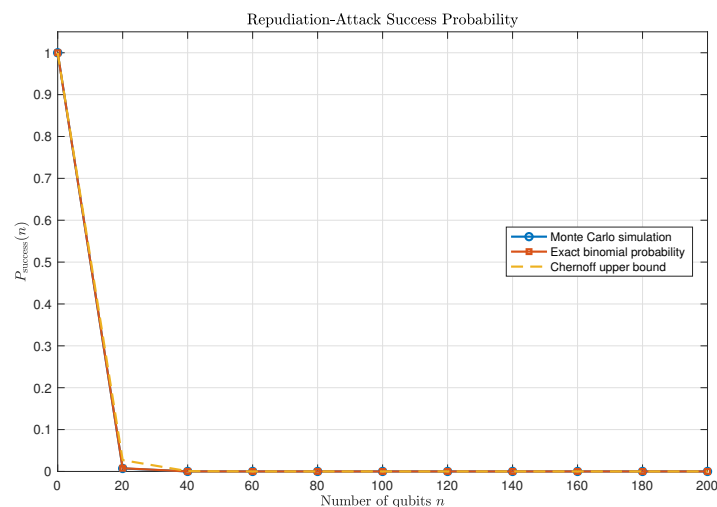


Figure 5. Attack success probability versus the number of qubits.

Finally, this constructed attack represents the strongest possible repudiation attempt, in which Alice deviates maximally in every step under her control. The analysis above covers three scenarios. If Bob is honest (Scenario 1), the protocol proceeds and Trent can trace the accepted message to Alice. If Bob's deviation prevents the protocol from proceeding (Scenario 2), Alice has no accepted signature to repudiate. If Bob's deviation inadvertently assists Alice (Scenario 3, probability $1/4$), the protocol proceeds and Trent's traceability still holds. Therefore, in all possible scenarios, Alice cannot successfully repudiate a signature that Bob has accepted.

5.2.2. Bob's attack simulation

We now analyze Bob's forgery attack under the assumption that Alice follows the protocol honestly. This corresponds to the standard unforgeability game in Section 4.3. If Alice were also dishonest, the analysis of Scenario 2 and Scenario 3 in the previous subsection applies symmetrically: Alice's deviation would either cause the protocol to abort or, with small probability, inadvertently assist Bob. In either case, Bob's forgery success remains bounded by the same arguments.

Bob's attack description. Bob has two attack opportunities. In the first stage (during protocol execution, Step (S3)-(S4)), Bob possesses $|p_2\rangle$ and $U(X_1)|p_1\rangle$. He may apply a non-identity operation G on $U(X_1)|p_1\rangle$ before teleporting it to Trent in Step (S4), or modify the plaintext copy $|p_2\rangle$. In the second stage (after protocol completion, Step (S7)), Bob holds $|p_2\rangle$ and $U(X_3)|p_1\rangle$, and may attempt to produce a forged signature.

Verification phase Analysis. For the first stage, if Bob applies $G \neq I$, the state reaching Trent becomes $U(X_2)GU(X_1)|p_1\rangle$. After Trent applies $U(X_1)$ and $U(X_2)$ in Step (S5), the recovered state is $U(X_1)GU(X_1)|p_1\rangle$ (up to a global phase). Trent compares this with $|p_3\rangle$ via the swap test. Since $|p_3\rangle$ was encrypted by Alice with K'_{AT} , Bob cannot alter it. The swap test detects the mismatch with probability at least $1 - [(1 + F_G)/2]^\lambda$, where F_G is the fidelity between the tampered state and the reference state. For orthogonal states ($F_G = 0$), this reduces to $1 - 2^{-\lambda}$. If Bob modifies $|p_2\rangle$ instead, the swap test in Step (S7) detects the mismatch with the same probability guarantee.

For the second stage, after recovering $|p_1\rangle$ and verifying it against $|p_2\rangle$, Bob already holds the valid message. To forge a different message, Bob would need to produce a new $|p^*\rangle$ and corresponding evidence that Trent accepts as originating from Alice. However, Trent retains the encrypted record $E_{K'_{AT}}|p_3\rangle$ and the particle pairs from both Alice and Bob, all protected by K_{AT} that Bob does not possess. Bob's knowledge of X_2 and X_3 provides no advantage, as these are random Bell measurement outcomes independent of the message content.

In summary, the above analysis shows that neither opportunity allows Bob to forge Alice's signature with non-negligible probability. This is consistent with the formal unforgeability proof in Section 4.3.

5.3. Numerical simulations

To complement the analytical security discussion, we performed numerical and hardware evaluations of the verification model. Reference implementations are provided in Appendices B and C, with complete workflows available in the supplementary material.

Figure 6 summarizes the batch-mode performance and resource overhead. Panel (a) shows the detection probability as a function of the number of verification instances m for fidelity values F from 0 to 0.99. The analytical curves agree with Monte Carlo simulations across the tested range. For small and moderate F , the detection probability increases rapidly with m ; however, when F approaches unity, such as $F = 0.99$, the detection rate remains limited even at $m = 40$, reflecting the inherent difficulty of distinguishing near-identical states. Panel (b) shows that as the batch parameter λ grows, the acceptance probabilities of the modeled tampering, impersonation, record-tampering, and replay-mismatch scenarios generally decrease, while the honest acceptance probability remains high. Alice scenario 1 remains statistically close to honest execution in this consistency-only diagnostic. The suppression is quantitative rather than absolute: the modeled attack-acceptance probabilities become small but do not necessarily reach zero for finite batch sizes. Panel (c) compares the exact binomial probability, the seeded Monte Carlo estimate, and the Chernoff upper bound for the separate statistical model of Alice's repudiation behavior. Panel (d) quantifies the cumulative physical-resource scaling versus the number

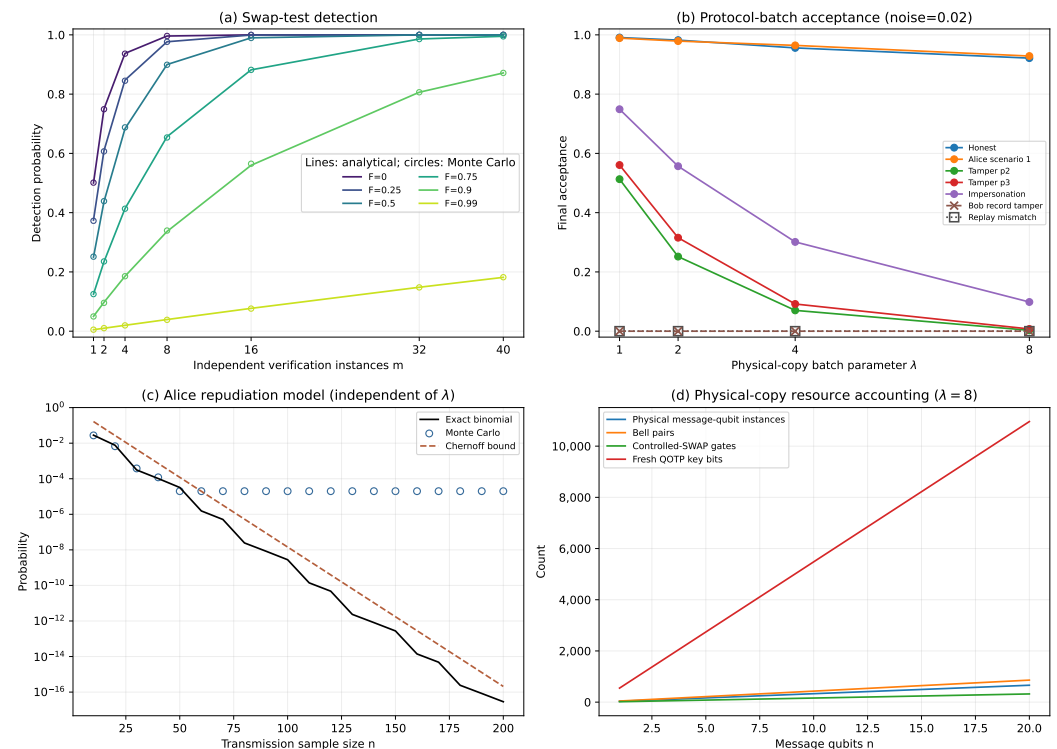


Figure 6. Protocol-batch performance and resource accounting. (a) Swap-test detection probability versus the number m of independent Swap-test comparisons for several fidelity values F . Lines are analytical results; circles are Monte Carlo simulations. (b) Final acceptance probability under honest execution and multiple attack scenarios versus the physical-copy batch parameter λ , under a simplified state-level noise model with noise parameter 0.02. Honest execution and Alice scenario 1 remain statistically close in this consistency-only diagnostic. (c) Alice’s repudiation probability versus the transmission sample size n under the independent binomial model. Exact binomial, Monte Carlo, and Chernoff-bound results are compared. (d) Cumulative physical-resource counts versus message size n for fixed batch parameter $\lambda = 8$.

of message qubits for a fixed batch parameter $\lambda = 8$, under the stated resource-counting definitions.

We note three limitations of the numerical evaluation. First, the $1 - 2^{-m}$ detection formula applies only to orthogonal inputs with $F = 0$; for high-fidelity cases such as $F = 0.99$, the detection probability grows much more slowly with m , and this should not be interpreted as a general closed-form detection law. Second, the batch-level curves are evaluated in the physical-copy batch model under a simplified state-level noise model, in which each relevant state is subjected to a fixed noise level. This model is not a device-level noise simulation and does not correspond to independent protocol instances. Under nonzero noise, even the honest acceptance probability decreases with λ , because every selected comparison must pass. Third, the binomial curve for Alice’s repudiation probability in Panel (c) is a statistical illustration under an independent binomial model; it does not by itself establish non-repudiation, which relies on the full protocol records and the adversarial analysis of Section 4.3.

Figure 7 presents circuit-level hardware evaluation results obtained from IBM Quantum devices and local simulations. Panels (a) and (b) show that quantum teleportation and the strengthened-QOTP roundtrip achieve correct-output probabilities mostly above 0.94 across different input states and keys, indicating basic feasibility despite noticeable hardware noise. Panel (c) reveals a significant gap between ideal/simulated and real-hardware Swap-test detection, highlighting the sensitivity of this operation to current-device noise.

IBM Quantum hardware results and local Aer reference

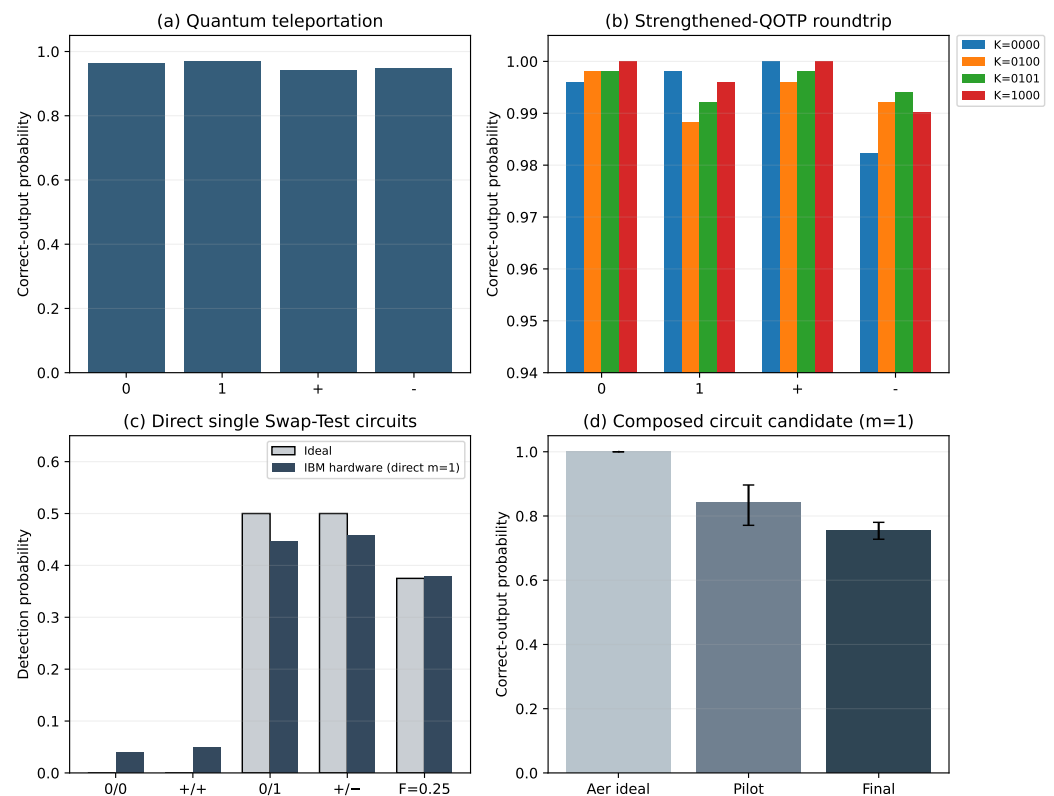


Figure 7. Circuit-level hardware evaluation of core quantum primitives on IBM Quantum devices and local simulations. (a) Correct-output probability for quantum teleportation under different input states. (b) Correct-output probability for the strengthened-QOTP roundtrip with various keys. (c) Detection probability for direct single Swap-test circuits, comparing ideal, local Aer simulation, and real hardware results. (d) Correct-output probability comparison for a composed circuit-level candidate corresponding to a single verification instance ($m = 1$), evaluated using IBM Quantum hardware and a local Aer reference.

Panel (d) further shows that an optimized composed circuit candidate with batch parameter $m = 1$ performs acceptably on hardware, albeit below simulator levels. These hardware runs are circuit-level benchmarks of the protocol's core primitives; they do not constitute a full implementation of the complete $(5\lambda + 3)$ -Bell-pair physical-copy batch protocol. In particular, the Swap-test gap between simulation and hardware indicates that direct deployment of the full batch protocol would require either noise suppression or additional error mitigation, and the present experiments should not be taken as an end-to-end hardware demonstration.

Overall, these evaluations show that the verification behavior is consistent with the fidelity-based analysis for the tested instances, that the batch structure and resource accounting follow the stated numerical model, and that the selected circuit-level quantum operations can be executed on current noisy quantum devices at a small scale.

6. Conclusion

This paper addressed the structural security challenges that arise when quantum search primitives are directly embedded into signature workflows. We identified structural vulnerabilities in a Grover-based quantum signature scheme and demonstrated that these vulnerabilities enable complete key recovery, Alice's disavowal, and Bob's existential forgery via man-in-the-middle attacks. To address these weaknesses, we proposed an

arbitrated quantum signature scheme based on quantum teleportation and a strengthened quantum one-time pad. We emphasize that the scheme targets quantum messages with known classical descriptions, and is not applicable to arbitrary unknown quantum states. The security of the proposed scheme was analyzed through a formal adversarial model with well-defined security games for unforgeability and non-repudiation, and its resilience against replay, intercept-and-resend, man-in-the-middle, entanglement, and collective or coherent attacks was examined. Numerical simulations illustrated the behavior of the swap test as a verification primitive and the linear scaling of resource consumption.

Beyond signature schemes, future work may further extend quantum search algorithms to other quantum cryptographic protocols or multi-party quantum communication scenarios. Potential directions include routing optimization, repeater selection, and network diagnostics, where quantum speedups hold promise for reducing latency and enhancing resource efficiency. These avenues warrant further investigation within the context of future quantum networks.

Acknowledgements

This work is supported by the National Natural Science Foundation of China (Nos. 62272208, 62572221).

Appendix A MATLAB code of Fig. 5

The following MATLAB code generates Fig. 5.

```
% Clear workspace
>> %% Parameter setting
alpha = 0.05;
beta = 0.06;
p = 0.25 + alpha;
trials = 20000;
n_list = 0:20:200;
sim_result = zeros(size(n_list));
theory_result = zeros(size(n_list));
for idx = 1:length(n_list)
    n = n_list(idx);
    threshold = floor(n * beta);
    success_case = 0;
    for t = 1:trials
        k = binornd(n,p);
        if(k <= threshold)
            success_case = success_case + 1;
        end
    end
    sim_result(idx) = success_case / trials;
    term1 = beta * log(beta / p);
    term2 = (1 - beta) * log((1 - beta) / (1 - p));
    theory_result(idx) = exp(-n * (term1 + term2));
end
%% Plot figure
figure('Position',[100,100,640,400])
plot(n_list,sim_result,'o-','LineWidth',2,'DisplayName','Monte-Carlo
Numerical Simulation');
hold on;
plot(n_list,theory_result,'--','LineWidth',2,'DisplayName','Chernoff Bound
Theoretical Upper-bound');
xlabel('Number of qubits $n$', 'Interpreter','latex');
ylabel('$P_{\text{success}}(n)$', 'Interpreter','latex');
title('Attack Success Probability: Simulation vs Theoretical Results');
legend;
grid on;
>>
```

Appendix B Python Code for Fig. 6

The following is a simplified reference implementation that illustrates the core logic of the numerical experiments in Fig. 6; the complete implementation is provided in the supplementary material.

```

"""Runnable reference implementation for the numerical results in Figure 6."""
from __future__ import annotations
import math
from dataclasses import dataclass, field
from pathlib import Path
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

SEED = 20260903
SWAP_TRIALS, PROTOCOL_TRIALS = 50_000, 4_000
FIDELITIES = (0.0, 0.25, 0.5, 0.75, 0.9, 0.99)
M_VALUES = (1, 2, 4, 8, 16, 32, 40)
LAMBDA_VALUES = (1, 2, 4, 8)
SCENARIOS = (
    "honest", "alice_scenario_1", "tamper_p2", "tamper_p3",
    "impersonation_attempt", "bob_tampering_s6_record",
    "replay_record_mismatch",
)

def swap_accept_single(fidelity: float) -> float:
    fidelity = float(fidelity)
    if fidelity < -1e-12 or fidelity > 1.0 + 1e-12:
        raise ValueError("fidelity must be in [0, 1]")
    return (1.0 + float(np.clip(fidelity, 0.0, 1.0))) / 2.0

def swap_accept_all(fidelities) -> float:
    values = list(fidelities)
    if not values:
        raise ValueError("at least one fidelity is required")
    return float(np.prod([swap_accept_single(value) for value in values]))

def effective_fidelity(fidelity: float, noise: float) -> float:
    """Simplified state-level noise rule, not a hardware noise model."""
    if not 0.0 <= noise <= 1.0:
        raise ValueError("noise must be in [0, 1]")
    value = (1.0 - noise) * fidelity + noise / 2.0
    return float(np.clip(value, 0.0, 1.0))

def physical_copy_counts(lam: int) -> dict[str, int]:
    if not isinstance(lam, int) or lam < 1:
        raise ValueError("lambda must be a positive integer")
    return {"p1": 2 * lam + 1, "p2": lam, "p3": lam, "total": 4 * lam + 1}

def encrypted_object_specs(lam: int):
    """Yield (purpose, copy_id, bit_length) for every protected object."""
    for j in range(lam):
        yield "p3_key", f"p3[{j}]", 4
    for i in range(2 * lam + 1):
        yield "s3_bell_key", f"p1[{i}]", 4
        yield "x1_record_key", f"p1[{i}]", 8
        yield "s4_bell_key", f"p1[{i}]", 4
        yield "x2_record_key", f"p1[{i}]", 8
    for j in range(lam + 1):
        yield "s6_bell_key", f"returned_p1[{j}]", 4
        yield "x3_record_key", f"returned_p1[{j}]", 8

```

```

@dataclass
class KeyBlock:
    block_id: str
    bits: str
    consumed: bool = False

class FreshKeyPool:
    def __init__(self, rng: np.random.Generator):
        self.rng, self.blocks = rng, {}

    def allocate_and_consume(self, length: int) -> str:
        block_id = f"key-{len(self.blocks):05d}"
        bits = "".join(map(str, self.rng.integers(0, 2, length)))
        self.blocks[block_id] = KeyBlock(block_id, bits)
        block = self.blocks[block_id]
        if block.consumed:
            raise RuntimeError("key block reuse")
        block.consumed = True
        return block_id

@dataclass
class BatchTrace:
    lam: int
    p1_ids: list[int]
    p2_ids: list[int]
    p3_ids: list[int]
    key_ids: dict[str, str]
    s5_test: list[int] = field(default_factory=list)
    s5_remaining: list[int] = field(default_factory=list)
    s7_test: list[int] = field(default_factory=list)
    retained: int | None = None

def prepare_batch(lam: int, rng: np.random.Generator) -> BatchTrace:
    counts = physical_copy_counts(lam)
    pool, key_ids = FreshKeyPool(rng), {}
    for purpose, copy_id, length in encrypted_object_specs(lam):
        key_ids[f"{purpose}:{copy_id}"] = pool.allocate_and_consume(length)
    assert all(block.consumed for block in pool.blocks.values())
    return BatchTrace(
        lam, list(range(counts["p1"])), list(range(counts["p2"])),
        list(range(counts["p3"])), key_ids,
    )

def scenario_fidelity(scenario: str, stage: str) -> float:
    """Fidelities defining the selected numerical diagnostic cases."""
    if scenario in {"honest", "alice_scenario_1"}:
        return 1.0
    if scenario == "tamper_p2":
        return 0.0 if stage == "S7" else 1.0
    if scenario == "tamper_p3":
        return 0.0 if stage == "S5" else 1.0
    if scenario == "impersonation_attempt":
        return 0.5 if stage == "S5" else 1.0
    if scenario in SCENARIOS:
        return 1.0
    raise ValueError(f"unknown scenario: {scenario}")

def sample_comparisons(fidelity, noise, count, records_ok, rng) -> bool:
    probability = swap_accept_single(effective_fidelity(fidelity, noise))
    outcomes = [rng.random() < probability for _ in range(count)]
    return bool(records_ok and all(outcomes))

```

```

def execute_batch(batch: BatchTrace, scenario: str, noise: float, rng) -> bool:
    """Execute explicit S5/S7 sampling and one outcome per selected pair."""
    lam = batch.lam
    batch.s5_test = sorted(rng.choice(batch.p1_ids, lam, replace=False).tolist())
    tested = set(batch.s5_test)
    batch.s5_remaining = [index for index in batch.p1_ids if index not in tested]
    trent_accept = sample_comparisons(
        scenario_fidelity(scenario, "S5"), noise, lam,
        scenario != "replay_record_mismatch", rng,
    )
    if not trent_accept:
        return False
    positions = rng.choice(range(lam + 1), lam, replace=False)
    batch.s7_test = sorted(batch.s5_remaining[int(i)] for i in positions)
    tested = set(batch.s7_test)
    remaining = [index for index in batch.s5_remaining if index not in tested]
    if len(remaining) != 1:
        raise RuntimeError("S7 must retain exactly one p1 copy")
    batch.retained = remaining[0]
    bob_accept = sample_comparisons(
        scenario_fidelity(scenario, "S7"), noise, lam,
        scenario != "bob_tampering_s6_record", rng,
    )
    return bool(trent_accept and bob_accept)

def resource_row(n: int, lam: int) -> dict:
    counts = physical_copy_counts(lam)
    key_bits = n * sum(length for _, _, length in encrypted_object_specs(lam))
    return {
        "n": n, "lambda": lam,
        "total_message_copies": counts["total"] * n,
        "total_bell_pairs": (5 * lam + 3) * n,
        "total_controlled_swaps": 2 * lam * n,
        "fresh_qotp_key_bits": key_bits,
    }

def swap_data() -> pd.DataFrame:
    rows, rng = [], np.random.default_rng(SEED)
    for fidelity in FIDELITIES:
        for m in M_VALUES:
            p = swap_accept_single(fidelity)
            outcomes = rng.random((SWAP_TRIALS, m)) < p
            exact = swap_accept_all([fidelity] * m)
            rows.append({
                "fidelity": fidelity, "m": m,
                "analytical_detection": 1.0 - exact,
                "monte_carlo_detection": 1.0 - np.mean(np.all(outcomes, axis=1)),
            })
    return pd.DataFrame(rows)

def protocol_data() -> pd.DataFrame:
    rows = []
    for scenario_index, scenario in enumerate(SCENARIOS):
        for noise in (0.0, 0.02, 0.05):
            for lam in LAMBDA_VALUES:
                accepted = 0
                for trial in range(PROTOCOL_TRIALS):
                    seed = (SEED + scenario_index * 10_000_000
                        + int(noise * 1000) * 100_000
                        + lam * 10_000 + trial)
                    rng = np.random.default_rng(seed)
                    accepted += execute_batch(
                        prepare_batch(lam, rng), scenario, noise, rng

```

```

        )
        rows.append({
            "scenario": scenario, "noise": noise, "lambda": lam,
            "acceptance": accepted / PROTOCOL_TRIALS,
        })
    return pd.DataFrame(rows)

def exact_binomial(n: int, alpha=0.05, beta=0.25) -> float:
    p, limit = 0.25 + alpha, math.floor(n * beta)
    return sum(math.comb(n, k) * p**k * (1 - p)**(n-k)
               for k in range(limit + 1))

def chernoff(n: int, alpha=0.05, beta=0.25) -> float:
    p = 0.25 + alpha
    divergence = beta * math.log(beta / p)
    divergence += (1-beta) * math.log((1-beta) / (1-p))
    return math.exp(-n * divergence)

def alice_data() -> pd.DataFrame:
    rows, rng = [], np.random.default_rng(SEED)
    for n in range(10, 201, 10):
        samples = rng.binomial(n, 0.30, 50_000)
        rows.append({
            "n": n, "exact": exact_binomial(n),
            "monte_carlo": np.mean(samples <= math.floor(0.25*n)),
            "chernoff": chernoff(n),
        })
    return pd.DataFrame(rows)

def resource_data() -> pd.DataFrame:
    return pd.DataFrame([resource_row(n, lam)
                        for lam in LAMBDA_VALUES for n in range(1, 21)])

def plot_figure6(swap, protocol, alice, resources, output: Path) -> None:
    fig, axes = plt.subplots(2, 2, figsize=(11, 8))
    for fidelity in FIDELITIES:
        part = swap[swap.fidelity == fidelity]
        axes[0, 0].plot(part.m, part.analytical_detection)
        axes[0, 0].plot(part.m, part.monte_carlo_detection, "o", fillstyle="none")
        selected = protocol[protocol.noise == 0.02]
        for scenario in SCENARIOS:
            part = selected[selected.scenario == scenario]
            axes[0, 1].plot(part["lambda"], part.acceptance, "o-", label=scenario)
    axes[0, 1].legend(fontsize=7)
    axes[1, 0].semilogy(alice.n, alice.exact, label="Exact")
    axes[1, 0].semilogy(alice.n, alice.monte_carlo, "o", label="Monte Carlo")
    axes[1, 0].semilogy(alice.n, alice.chernoff, "--", label="Chernoff")
    part = resources[resources["lambda"] == 8]
    for column in ("total_message_copies", "total_bell_pairs",
                  "total_controlled_swaps", "fresh_qotp_key_bits"):
        axes[1, 1].plot(part.n, part[column], label=column)
    titles = ("(a) Swap-test detection", "(b) Protocol-batch acceptance",
              "(c) Alice repudiation model", "(d) Physical-copy resources")
    for axis, title in zip(axes.ravel(), titles):
        axis.set_title(title); axis.grid(alpha=0.25)
    axes[1, 0].legend(); axes[1, 1].legend(fontsize=7)
    fig.tight_layout(); fig.savefig(output, bbox_inches="tight"); plt.close(fig)

def main() -> None:
    output = Path("results"); data_dir = output / "data"
    figure_dir = output / "figures"
    data_dir.mkdir(parents=True, exist_ok=True)
    figure_dir.mkdir(parents=True, exist_ok=True)

```

```

swap, protocol = swap_data(), protocol_data()
alice, resources = alice_data(), resource_data()
swap.to_csv(data_dir / "swap_test_detection.csv", index=False)
protocol.to_csv(data_dir / "protocol_acceptance.csv", index=False)
alice.to_csv(data_dir / "alice_repudiation_probability.csv", index=False)
resources.to_csv(data_dir / "resource_scaling.csv", index=False)
plot_figure6(swap, protocol, alice, resources,
              figure_dir / "Figure6_protocol_batch.pdf")

if __name__ == "__main__":
    main()

```

Appendix C Python Code for Fig. 7

The following is a simplified reference implementation that illustrates the core logic of the numerical experiments in Fig. 7; the complete implementation is provided in the supplementary material.

```

"""Standalone circuit-level reference code for Figure 7.

The default execution constructs local circuits and never connects to IBM.
Existing IBM result files can be plotted with the optional '--data-dir'.
"""

from __future__ import annotations

import argparse
from math import sqrt
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from qiskit import ClassicalRegister, QuantumCircuit, QuantumRegister
from qiskit.qasm3 import dumps as qasm3_dumps

X = np.array([[0, 1], [1, 0]], dtype=complex)
Y = np.array([[0, -1j], [1j, 0]], dtype=complex)
Z = np.array([[1, 0], [0, -1]], dtype=complex)
T_STRENGTHENED = np.sqrt(1j / 3) * (X - Y + Z)

def encryption_unitary(key4: str) -> np.ndarray:
    """Return the strengthened-QOTP unitary for one four-bit block."""
    if len(key4) != 4 or set(key4) - {"0", "1"}:
        raise ValueError("key4 must contain four binary digits")
    a, b, c, d = map(int, key4)
    return (
        np.linalg.matrix_power(X, a)
        @ np.linalg.matrix_power(Z, b)
        @ T_STRENGTHENED
        @ np.linalg.matrix_power(X, c)
        @ np.linalg.matrix_power(Z, d)
    )

def prepare_state(circuit, qubit, state: str) -> None:
    if state == "1":
        circuit.x(qubit)
    elif state == "+":

```

```

        circuit.h(qubit)
    elif state == "-":
        circuit.x(qubit)
        circuit.h(qubit)
    elif state != "0":
        raise ValueError("state must be one of: 0, 1, +, -")

def return_to_zero_basis(circuit, qubit, state: str) -> None:
    if state in {"+", "-"}:
        circuit.h(qubit)
    if state in {"1", "-"}:
        circuit.x(qubit)

def prepare_bell_pair(circuit, sender, receiver) -> None:
    circuit.h(sender)
    circuit.cx(sender, receiver)

def coherent_teleportation(circuit, message, sender, receiver) -> None:
    """Deferred-measurement form of teleportation with coherent corrections."""
    circuit.cx(message, sender)
    circuit.h(message)
    circuit.cx(sender, receiver)
    circuit.cz(message, receiver)

def append_swap_test(circuit, ancilla, left, right) -> None:
    circuit.h(ancilla)
    circuit.cswap(ancilla, left, right)
    circuit.h(ancilla)

def append_qotp_roundtrip(circuit, qubit, key4: str) -> None:
    unitary = encryption_unitary(key4)
    circuit.unitary(unitary, [qubit], label="QOTP-enc")
    circuit.unitary(unitary.conj().T, [qubit], label="QOTP-dec")

def build_teleportation_circuit(state: str = "0") -> QuantumCircuit:
    circuit = QuantumCircuit(3, 1, name=f"teleport-{state}")
    prepare_state(circuit, 0, state)
    prepare_bell_pair(circuit, 1, 2)
    coherent_teleportation(circuit, 0, 1, 2)
    return_to_zero_basis(circuit, 2, state)
    circuit.measure(2, 0)
    return circuit

def build_qotp_roundtrip_circuit(
    key4: str = "0101", state: str = "0"
) -> QuantumCircuit:
    circuit = QuantumCircuit(1, 1, name="qotp-roundtrip")
    prepare_state(circuit, 0, state)
    append_qotp_roundtrip(circuit, 0, key4)
    return_to_zero_basis(circuit, 0, state)
    circuit.measure(0, 0)
    return circuit

def build_swap_test_circuit(
    left: str = "0", right: str = "1"

```

```

) -> QuantumCircuit:
    circuit = QuantumCircuit(3, 1, name="swap-test")
    prepare_state(circuit, 1, left)
    prepare_state(circuit, 2, right)
    append_swap_test(circuit, 0, 1, 2)
    circuit.measure(0, 0)
    return circuit

def expected_composed_output(state: str) -> str:
    """Qiskit displays classical bits in descending register order."""
    if state not in {"0", "1", "+", "-"}:
        raise ValueError("state must be one of: 0, 1, +, -")
    return "100" if state in {"1", "-"} else "000"

def correct_output_probability(counts: dict[str, int], valid: set[str]) -> float:
    """Sum counts over the explicitly valid logical output events."""
    normalized = {key.replace(" ", ""): value for key, value in counts.items()}
    shots = sum(normalized.values())
    if shots <= 0:
        raise ValueError("counts must contain at least one shot")
    return sum(normalized.get(key, 0) for key in valid) / shots

def build_composed_candidate(state: str = "0") -> QuantumCircuit:
    """One-copy circuit-level candidate; this is not the full batch protocol."""
    qreg = QuantumRegister(9, "q")
    creg = ClassicalRegister(3, "out")
    circuit = QuantumCircuit(qreg, creg, name="composed-candidate")
    for qubit in (0, 1, 2):
        prepare_state(circuit, qubit, state)
    for sender, receiver in ((3, 4), (5, 6), (7, 8)):
        prepare_bell_pair(circuit, sender, receiver)
    coherent_teleportation(circuit, 0, 3, 4)
    coherent_teleportation(circuit, 4, 5, 6)
    circuit.reset(0)
    append_swap_test(circuit, 0, 6, 2)
    coherent_teleportation(circuit, 6, 7, 8)
    circuit.reset(4)
    append_swap_test(circuit, 4, 8, 1)
    return_to_zero_basis(circuit, 8, state)
    circuit.measure(0, 0)
    circuit.measure(4, 1)
    circuit.measure(8, 2)
    circuit.metadata = {
        "verification_instances": 1,
        "valid_expected_outputs": [expected_composed_output(state)],
        "not_implemented": [
            "QKD",
            "authenticated classical channels",
            "network separation",
            "arbitration",
        ],
    }
    return circuit

def wilson_interval(
    successes: int,
    shots: int,
    z: float = 1.959963984540054,
) -> tuple[float, float]:

```

```

        if shots <= 0 or successes < 0 or successes > shots:
            raise ValueError("require 0 <= successes <= shots and shots > 0")
        estimate = successes / shots
        denominator = 1.0 + z * z / shots
        center = (estimate + z * z / (2.0 * shots)) / denominator
        variance = estimate * (1.0 - estimate) / shots
        variance += z * z / (4.0 * shots * shots)
        half_width = z * sqrt(variance) / denominator
        return center - half_width, center + half_width

def as_float(value) -> float:
    return float(str(value).strip())

def generate_figure7(data_dir: Path, output: Path) -> None:
    """Plot existing local and IBM results without any backend connection."""
    teleportation = pd.read_csv(data_dir / "teleportation_hardware.csv")
    qotp = pd.read_csv(data_dir / "qotp_hardware.csv")
    swap = pd.read_csv(data_dir / "swap_test_hardware.csv")
    aer = pd.read_csv(data_dir / "composed_candidate_aer.csv").iloc[0]
    pilot = pd.read_csv(data_dir / "composed_candidate_pilot.csv").iloc[0]
    final = pd.read_csv(data_dir / "composed_candidate_final.csv").iloc[0]

    fig, axes = plt.subplots(2, 2, figsize=(11, 8))
    states = ["0", "1", "+", "-"]
    tele = teleportation.set_index("input_state").reindex(states)
    axes[0, 0].bar(states, tele["correct_output_probability"])
    axes[0, 0].set(
        title="(a) Quantum teleportation",
        ylabel="Correct-output probability",
        ylim=(0, 1.05),
    )

    positions = np.arange(4)
    keys = sorted(qotp["key"].astype(str).unique())[:4]
    for index, key in enumerate(keys):
        part = qotp[qotp["key"].astype(str) == key]
        part = part.set_index("input_state").reindex(states)
        axes[0, 1].bar(
            positions + (index - 1.5) * 0.18,
            part["correct_output_probability"],
            0.18,
            label=f"K={int(key):04d}",
        )
    axes[0, 1].set_xticks(positions, states)
    axes[0, 1].set(
        title="(b) Strengthened-QOTP roundtrip",
        ylabel="Correct-output probability",
        ylim=(0.94, 1.005),
    )
    axes[0, 1].legend(fontsize=8)

    cases = [
        "identical_0_0",
        "identical_plus_plus",
        "orthogonal_0_1",
        "orthogonal_plus_minus",
        "fixed_overlap_F_0.25",
    ]
    selected = swap[
        (swap["execution"] == "Final") & (swap["lambda"] == 1)
    ]

```

```

selected = selected.set_index("case").loc[cases]
ideal = (1.0 - selected["fidelity"].to_numpy()) / 2.0
axes[1, 0].bar(np.arange(5) - 0.18, ideal, 0.36, label="Ideal")
axes[1, 0].bar(
    np.arange(5) + 0.18,
    selected["single_test_detection_probability"],
    0.36,
    label="IBM hardware",
)
axes[1, 0].set_xticks(
    range(5), ["0/0", "+/+ ", "0/1", "+/- ", "F=.25"]
)
axes[1, 0].set(
    title="(c) Direct single Swap Tests (m=1)",
    ylabel="Detection probability",
    ylim=(0, 0.65),
)
axes[1, 0].legend(fontsize=8)

values = [
    as_float(aer["success_probability"]),
    as_float(pilot["success_probability"]),
    as_float(final["success_probability"]),
]
axes[1, 1].bar(["Aer ideal", "Pilot", "Final"], values)
axes[1, 1].set(
    title="(d) Composed circuit-level candidate (m=1)",
    ylabel="Correct-output probability",
    ylim=(0, 1.05),
)
for axis in axes.ravel():
    axis.grid(axis="y", alpha=0.2)
fig.tight_layout()
output.parent.mkdir(parents=True, exist_ok=True)
fig.savefig(output, dpi=300, bbox_inches="tight")
plt.close(fig)

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--state", choices=["0", "1", "+", "-"], default="+")
    parser.add_argument("--output-dir", type=Path, default=Path("results"))
    parser.add_argument("--data-dir", type=Path)
    args = parser.parse_args()

    args.output_dir.mkdir(parents=True, exist_ok=True)
    circuit = build_composed_candidate(args.state)
    qasm_path = args.output_dir / "composed_candidate.qasm"
    qasm_path.write_text(qasm3_dumps(circuit), encoding="utf-8")
    print(
        f"qubits={circuit.num_qubits}, "
        f"depth={circuit.depth()}, size={circuit.size()}"
    )
    if args.data_dir is not None:
        generate_figure7(
            args.data_dir,
            args.output_dir / "Figure7_hardware_scope.pdf",
        )

if __name__ == "__main__":
    main()

```

References

1. Shi, R. H.; Feng, Y. Y.; Shi, J. J. Arbitrated Quantum Signature Scheme with Quantum Walks on Regular Graphs. *J. Electron. Inf. Technol.* **2020**, *42*, 89–97.
2. Shang, T.; Hou, P. L.; Liu, Y. C.; Xiong, K. Y.; Liu, J. W. Arbitrated Quantum (t,n) Threshold Group Signature Scheme Based on Multi-coin Quantum Walk. *Sci. Sin. Phys. Mech. Astron.* **2024**, *54*, 240316.
3. Shi, R. H.; Zhang, M. W. Privacy-preserving Quantum Sealed-bid Auction Based on Grover's Search Algorithm. *Sci. Rep.* **2019**, *9*, 7626.
4. Yoon, C. S.; Kang, M. S.; Lim, J. I.; Yang, H. J. Quantum Signature Scheme Based on a Quantum Search Algorithm. *Phys. Scr.* **2015**, *90*, 015103.
5. Grover, L. K. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the 28th Annual ACM Symposium on the Theory of Computing*, Philadelphia, PA, USA, 22–24 May 1996; pp. 212–219.
6. Yoon, C. S.; Hong, C. H.; Kang, M. S.; Choi, J. W.; Yang, H. J. Quantum Asymmetric Key Crypto Scheme Using Grover Iteration. *Sci. Rep.* **2023**, *13*, 3810.
7. Lu, D.; Li, Z.; Yu, J.; Han, Z. A Verifiable Arbitrated Quantum Signature Scheme Based on Controlled Quantum Teleportation. *Entropy* **2022**, *24*, 111.
8. Zeng, G.; Keitel, C. H. Arbitrated-Quantum Signature Scheme. *Phys. Rev. A* **2002**, *65*, 042312.
9. Li, Q.; Chan, W. H.; Long, D. Y. Arbitrated Quantum Signature Scheme Using Bell States. *Phys. Rev. A* **2009**, *79*, 054307.
10. Pang, Z.; Xiang, H. A Simplified Arbitrated Quantum Signature Protocol Without Entanglement: Design Premise and Security Analysis. *Opt. Commun.* **2025**, *595*, 132315.
11. Zou, X.; Qiu, D. Security Analysis and Improvements of Arbitrated Quantum Signature Schemes. *Phys. Rev. A* **2010**, *82*, 042325.
12. Yang, Y. G.; Zhou, Z.; Teng, Y. W.; Wen, Q. Y. Arbitrated Quantum Signature with an Untrusted Arbitrator. *Eur. Phys. J. D* **2011**, *61*, 773–778.
13. Li, Q.; Li, C.; Long, D.; Chan, W. H.; Wang, C. Efficient Arbitrated Quantum Signature and Its Proof of Security. *Quantum Inf. Process.* **2013**, *12*, 2427–2439.
14. Choi, J. W.; Chang, K. Y.; Hong, D. Security Problem on Arbitrated Quantum Signature Schemes. *Phys. Rev. A* **2011**, *84*, 062330.
15. Zhang, K. J.; Zhang, W. W.; Li, D. Improving the Security of Arbitrated Quantum Signature Against the Forgery Attack. *Quantum Inf. Process.* **2013**, *12*, 2655–2669.
16. Kim, T.; Choi, J. W.; Jho, N. S.; Lee, S. Quantum Messages with Signatures Forgeable in Arbitrated Quantum Signature Schemes. *Phys. Scr.* **2015**, *90*, 025101.
17. Boykin, P. O.; Roychowdhury, V. Optimal Encryption of Quantum Bits. *Phys. Rev. A* **2003**, *67*, 042317.
18. Leung, D. W. Quantum Vernam Cipher. arXiv:quant-ph/0012077, 2000.
19. Bennett, C. H.; Brassard, G. Quantum Cryptography: Public Key Distribution and Coin Tossing. In *Proceedings of IEEE International Conference on Computers, Systems and Signal Processing*, Bangalore, India, 9–12 December 1984; pp. 175–179.
20. Ekert, A. K. Quantum Cryptography Based on Bell's Theorem. *Phys. Rev. Lett.* **1991**, *67*, 661–663.
21. Bennett, C. H. Quantum Cryptography Using Any Two Nonorthogonal States. *Phys. Rev. Lett.* **1992**, *68*, 3121–3124.
22. Mayers, D. Unconditional Security in Quantum Cryptography. *J. ACM* **2001**, *48*, 351–406.
23. Lo, H. K. A Simple Proof of the Unconditional Security of Quantum Key Distribution. *J. Phys. A: Math. Gen.* **2001**, *34*, 6957.
24. Lu, H.; Guo, G. C. Teleportation of a Two-Particle Entangled State via Entanglement Swapping. *Phys. Lett. A* **2000**, *276*, 209–212.
25. Lo, H. K.; Chau, H. F. Unconditional Security of Quantum Key Distribution over Arbitrarily Long Distances. *Science* **1999**, *283*, 2050–2056.
26. Shor, P. W.; Preskill, J. Simple Proof of Security of the BB84 Quantum Key Distribution Protocol. *Phys. Rev. Lett.* **2000**, *85*, 441–444.

27. Abruzzo, S.; Bratzik, S.; Bernardes, N. K.; Kampermann, H.; van Loock, P.; Bruß, D. Quantum Repeater and Quantum Key Distribution: Analysis of Secret-Key Rates. *Phys. Rev. A* **2013**, *87*, 052315. 1810 1811 1812
28. Barnum, H.; Crépeau, C.; Gottesman, D.; Smith, A.; Tapp, A. Authentication of Quantum Messages. In *Proceedings of the 43rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Vancouver, BC, Canada, 16–19 November 2002; pp. 449–458. 1813 1814 1815